RTL Design Sherpa

RTL AMBA AXI4-Lite Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXIL4 (AXI4-Lite) Modules

Location: `rtl/amba/axil4/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

The AXIL4 subsystem is a complete implementation of the ARM AMBA AXI4-Lite protocol — the simplified subset of AXI4 built for control-register access, with a reduced signal count and single-beat transactions only. This is the bus you reach for when a register needs an interface, not when data needs a firehose.

1.1.1 Key Features

- **AXI4-Lite Protocol:** Simplified subset (no burst, no ID, no QoS)
- **Single-Beat Transactions:** Always 1 transfer per transaction
- **Reduced Signals:** 40-50% fewer signals vs full AXI4
- **Buffered Interfaces:** Elastic buffering via gaxi_skid_buffer
- **Monitoring Infrastructure:** Integrated transaction monitoring
- **Clock Gating Variants:** Power-optimized versions
- **Typical Use:** Control registers, CSRs, peripheral access

1.1.2 Core Master/Slave Modules

Module	Description	Documentation	Status
axil4_master_rd	AXIL4 read master with buffered channels	axil4_master_rd.md	Documented
axil4_master_wr	AXIL4 write master with buffered channels	axil4_master_wr.md	Documented
axil4_slave_rd	AXIL4 read slave with buffered channels	axil4_slave_rd.md	Documented
axil4_slave_wr	AXIL4 write slave with buffered channels	axil4_slave_wr.md	Documented

1.1.3 Monitor Modules (Verification)

Monitor wrappers live in `rtl/amba/monitor/` (not `rtl/amba/axil4/`). They are dedicated AXIL4 wrappers — there is no `IS_AXI`-style parameter overload of the AXI4 wrappers — and they share `axi_monitor_base` and the 128-bit packet format with the AXI4 wrappers.

Module	Description	Documentation	Status
axil4_master_rd_mon	Read master with integrated monitoring	axil4_master_rd_mon.md	Documented
axil4_master_wr_mon	Write master with integrated monitoring	axil4_master_wr_mon.md	Documented
axil4_slave_rd_mon	Read slave with integrated monitoring	axil4_slave_rd_mon.md	Documented
axil4_slave_	Write slave with	axil4_slave_wr_mon.md	Documented

Module	Description	Documentation	Status
wr_mon	integrated monitoring		

1.1.4 Clock-Gated Variants

All clock-gated variants documented in: axil4_clock_gating_guide.md

Module	Base Module	Status
axil4_master_rd_cg	axil4_master_rd	Documented
axil4_master_wr_cg	axil4_master_wr	Documented
axil4_slave_rd_cg	axil4_slave_rd	Documented
axil4_slave_wr_cg	axil4_slave_wr	Documented
axil4_master_rd_mon_cg	axil4_master_rd_mon	Documented
axil4_master_wr_mon_cg	axil4_master_wr_mon	Documented
axil4_slave_rd_mon_cg	axil4_slave_rd_mon	Documented
axil4_slave_wr_mon_cg	axil4_slave_wr_mon	Documented

1.2 Functional Description

1.2.1 Simplified vs Full AXI4

AXI4-Lite keeps the AXI4 handshake model and drops everything a register interface doesn't need:

Feature	AXI4	AXI4-Lite
Burst Support	Yes (1-256 beats)	No (always 1 beat)
ID Signals	ARID/AWID/RID/BID	None
Out-of-Order	Supported via ID	Not supported
Data Width	8-1024 bits	32 or 64 bits
QoS/Region	Supported	Not supported
Lock/Cache	Full support	None (no AxLOCK,

Feature	AXI4	AXI4-Lite
		no AxCACHE)
Signal Count	~40-50 signals	~20-25 signals
Typical Use	High-bandwidth data	Control registers

1.2.2 Channel Architecture

AXI4-Lite defines five independent channels. Any one transaction uses only a subset of them: a write uses three (AW, W, B) and a read uses two (AR, R). The read and write channel sets are fully independent and can be in flight concurrently.

Write Transaction (3 channels): 1. **AW (Write Address)** - Address and protection 2. **W (Write Data)** - Data and byte strobes 3. **B (Write Response)** - Completion status

Read Transaction (2 channels): 1. **AR (Read Address)** - Address and protection 2. **R (Read Data)** - Data and response

1.2.3 Key Signals (Simplified)

Address Channels (AR/AW): - `ARADDR/AWADDR` - Byte address - `ARPROT/AWPROT` - Protection attributes (3 bits) - `ARVALID/AWVALID`, `ARREADY/AWREADY` - Handshake

Data Channels (R/W): - `RDATA/WDATA` - Transfer data (32 or 64 bits) - `RRESP/BRESP` - Response (OKAY, SLVERR, DECERR) - `WSTRB` - Write strobes (byte lane enables) - `RVALID/WVALID`, `RREADY/WREADY` - Handshake

---

1.3 Usage Examples

1.3.1 Basic Read Master

```
axil4_master_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),    // 2 entries
    .SKID_DEPTH_R(4)      // 4 entries
) u_rd_master (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),

    // Frontend interface (from CPU)
    .fub_araddr     (cpu_araddr),
    .fub_arprot     (cpu_arprot),
    .fub_arvalid    (cpu_arvalid),
```

```
    .fub_arready    (cpu_arready),

    .fub_rdata      (cpu_rdata),
    .fub_rresp      (cpu_rresp),
    .fub_rvalid     (cpu_rvalid),
    .fub_rready     (cpu_rready),

    // Master interface (to interconnect)
    .m_axil_araddr  (periph_araddr),
    // ... rest of AR/R signals

    .busy           (rd_busy)
);
```

1.3.2 Basic Write Slave

```
axil4_slave_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_wr_slave (
    .aclk             (axi_clk),
    .aresetn          (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_awaddr    (interconnect_awaddr),
    // ... rest of AW/W/B signals

    // Backend interface (to register block)
    .fub_awaddr       (regfile_awaddr),
    // ... rest of AW/W/B signals

    .busy             (wr_busy)
);
```

1.3.3 Monitor Integration

```
axil4_master_rd_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(1),
    .AGENT_ID(10),
    .MAX_TRANSACTIONS(8),    // Reduced for AXIL (axi4_*_mon defaults
to 16)
    .ENABLE_FILTERING(1)
) u_rd_mon (
```

```
    .aclk                   (axi_clk),
    .aresetn                (axi_resetn),

    // AXIL interfaces
    .fub_axil_araddr        (cpu_araddr),
    // ... rest of AR/R signals

    // Monitor configuration
    .cfg_monitor_enable     (1'b1),
    .cfg_error_enable       (1'b1),
    .cfg_timeout_enable     (1'b1),
    .cfg_perf_enable        (1'b0),  // Avoid congestion
    .cfg_timeout_cycles     (16'd1000),

    // Filtering masks - see note below on polarity
    .cfg_axi_pkt_mask       (16'b1111_1111_1111_0110),  // Keep Error
+ Timeout
    // ... rest of masks

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),
    .monbus_timestamp       (mon_timestamp),

    // Status
    .busy                   (rd_busy),
    .active_transactions    (rd_active),
    .error_count            (rd_errors)
);
```

`cfg_axi_pkt_mask` polarity and indexing. The mask is 16 bits because it is indexed by the 4-bit `packet_type` field of the monitor packet — one mask bit per packet type — not by the 128-bit packet width. A **set bit drops** that packet type (`pkt_drop = cfg_axi_pkt_mask[pkt_type]` in `axi_monitor_filtered`). So `16'b1111_1111_1111_0110` clears bit 0 (`PktTypeError`) and bit 3 (`PktTypeTimeout`) to let those through and drops everything else. See the Monitor Packet Specification for the packet-type enum.

1.4 Timing Characteristics

1.4.1 Throughput

Configuration	Rate	Notes
Single-beat	1 txn/cycle	When buffers not

Configuration	Rate	Notes
read/write		stalled
Typical peripheral	1 txn/N cycles	N = backend response time

1.4.2 Latency

Latencies below are the module's own buffering overhead and **exclude** slave response time. Each channel traversed costs one clock in its skid buffer.

Path	Cycles	Notes
AR → R (single)	Slave latency + 2	1 clock in the AR buffer, 1 in the R buffer
AW+W → B (single)	Slave latency + 2	AW and W buffers traverse in parallel, then the B buffer
Buffer overhead	+1 per channel	Skid buffer latency

The write figure assumes the slave accepts AW and W in the same cycle. If the slave's AWREADY and WREADY assert in different cycles, the write costs the later of the two handshakes instead.

1.4.3 Resource Usage

The figures below are **order-of-magnitude estimates only**. They are not measured synthesis results: no target device, tool version, or optimization setting is attached to them. Treat them as relative sizing between the variants, and run synthesis for your own device before budgeting area.

Module Type	LUTs	FFs	BRAM	Notes
Master/Slave (32-bit)	~200	~150	0	Per direction (rd/wr)
Monitor (+mon)	thousands	**>5,000**	0	At this family's default `MAX_TRANSACTIONS = 8`. Structural floor counted from the RTL: transaction

Module Type	LUTs	FFs	BRAM	Notes
				table 285 b x 8 = 2,280 FF, the reporter's local copy another 2,280, timeout timers 384, threshold latency 256
Clock-gated (+cg)	not measured	+5	0	Counted from the RTL: `r_wakeup` (1 FF, `amba_clock_gate_ctrl`) + `r_idle_counter` (`IDLE_CNTR_WIDTH`, default 4, `clock_gate_ctrl`). Scales as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG is a latch/BUFGCE, not a fabric flop

vs AXI4: roughly 40-50% smaller, on the same estimate basis (no ID tracking, no burst counters, simpler protocol). This is an estimate, not a measured comparison against the AXI4 modules.

1.5 Design Notes

1.5.1 When to Use AXI4-Lite

Use AXI4-Lite for: - Control and status registers (CSRs) - Peripheral configuration interfaces - Low-bandwidth control paths - Resource-constrained designs - Simpler protocol requirements

Use full AXI4 for: - High-bandwidth data transfers - Burst transactions required - Out-of-order support needed - DMA engines, memory controllers - Streaming data

1.5.2 Buffer Depth Guidelines

The `SKID_DEPTH_*` parameters are passed straight through to the `DEPTH` parameter of `gaxi_skid_buffer`, which is the **literal entry count**, not an exponent. `SKID_DEPTH_AR = 2` means a 2-entry buffer. `gaxi_skid_buffer` constrains `DEPTH` to one of 2..8 inclusive.

Address Channels (AR/AW): - Default: 2 entries - sufficient for most register access - Increase for high-latency address decode

Data Channels (R/W): - Default: 4 entries for R, 2 entries for W - R channel typically deeper (accommodate backend latency) - W channel can be shallower (single-beat)

Response Channel (B): - Default: 2 entries - responses are single-beat - Rarely needs adjustment

1.5.3 Design Patterns

1.5.3.1 Pattern 1: Buffered Master/Slave

All core modules use `gaxi_skid_buffer`: - Decouples frontend and backend timing - Configurable depth per channel - 1-cycle latency overhead - Full backpressure handling

1.5.3.2 Pattern 2: Monitor Integration

Monitor modules combine functional core with verification: - Uses shared **axi_monitor_filtered** (rtl/amba/monitor/) - 2-level filtering (packet-type masks, then per-event-code masks; `err_select` is reserved and routes nothing) - 128-bit monitor bus + 64-bit side-band timestamp - Simplified for AXIL (MAX_TRANSACTIONS=8, vs 16 for the AXI4 wrappers)

1.5.3.3 Pattern 3: Clock Gating

Clock-gated variants (`*_cg`) add power management: - Dynamic gating based on channel valid activity plus the base module's `busy` - Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. AXIL4 is single-stage, so the first usable gated-clock edge arrives 2 clocks after activity asserts — see the Clock-Gated Variants Guide - Better power savings than AXI4 (bursty register access)

1.5.4 Known Limitations

- **No clock domain crossing.** All modules are fully synchronous to `aclk`. Cross domains with `gaxi_skid_buffer_async` or a CDC FIFO outside these modules.
- **No address decode or protection enforcement.** The modules are transport only: `AxPROT` is carried through untouched and never checked. Address decode belongs to the interconnect or the register block.
- **No error injection or response rewriting.** `RRESP` / `BRESP` pass through from the backend unmodified.
- **No write-data reordering or AW/W matching.** The AW and W channels are buffered independently; nothing in these modules pairs an address with its data. A backend that requires AW before W must enforce that itself.
- **Unaligned and narrow accesses are the backend's problem.** AXI4-Lite has no `AxSIZE`; partial-word access is expressed only through `WSTRB`, which the backend must honor.
- **`gaxi_skid_buffer` constrains `SKID_DEPTH_*` to 2..8 inclusive.** Other values are not supported.
- **Clock gating is ASIC-oriented.** `clock_gate_ctrl` instantiates an ICG cell; on FPGA this does not map to true clock gating. There is no scan/test bypass port — set `cfg_cg_enable = 0` to disable gating.

1.5.5 Terminology

Term	Meaning
FUB	Functional Unit Block — the design-internal logic on the far side of the module from the external AXI4-Lite bus. FUB-side ports are prefixed `fub_`.
Skid buffer	A small elastic FIFO (`gaxi_skid_buffer`) placed in a valid/ready channel. It registers the handshake so neither side sees a combinational path to the other, at the cost of one clock of latency.
CG	Clock Gating. The `_cg` module suffix denotes the clock-gated variant.
MonBus	The 128-bit monitor packet bus (plus 64-bit

Term	Meaning
	side-band timestamp) emitted by the `_mon` wrappers.

1.6 Related Modules

- **rtl-amba Overview** - Complete AMBA subsystem architecture
- **AXI4 Modules** - Full AXI4 protocol (comparison)
- **APB Modules** - Even simpler peripheral bus
- **GAXI Modules** - Generic AXI utilities (buffers, FIFOs)

1.7 Testing

All AXIL4 modules are verified using CocoTB-based testbenches:

```
# Run all AXIL4 tests
pytest val/amba/test_axil4*.py -v

# Run specific module tests
pytest val/amba/test_axil4_master_rd.py -v
pytest val/amba/test_axil4_slave_wr.py -v
pytest val/amba/test_axil4_master_rd_mon.py -v

# Run with waveforms
pytest val/amba/test_axil4_master_rd.py --vcd=waves.vcd -v
```

1.8 References

1.8.1 Protocol Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification
- Chapter 4: AXI4-Lite

1.8.2 Configuration and Integration

- **AXI Monitor Configuration Guide** - Monitor setup strategies
- **Monitor Packet Specification** - 128-bit packet plus 64-bit side-band timestamp

1.8.3 Source Code

- RTL: `rtl/amba/axil4/`
- Monitor wrappers: `rtl/amba/monitor/`
- Tests: `val/amba/test_axil4*.py`
- Framework: `bin/TBClasses/components/axil4/`
- Shared Infrastructure: `rtl/amba/monitor/` (monitor base and filter), `rtl/amba/gaxi/` (gaxi_skid_buffer)

Last Updated: 2026-07-19

1.9 Navigation

- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

2 AXIL4 Clock-Gated Variants Guide

Location: `rtl/amba/axil4/*_cg.sv` **Status:** Production Ready

2.1 Overview

All AXIL4 modules have clock-gated (`_cg`) variants that add power management through dynamic clock gating. Identical architecture to **AXI4 Clock Gating** but optimized for AXI4-Lite's simpler protocol.

2.1.1 Available Clock-Gated Modules

Module	Base Module	Description
`axil4_master_rd_cg`	axil4_master_rd	Clock-gated read master
`axil4_master_wr_cg`	axil4_master_wr	Clock-gated write master
`axil4_slave_rd_cg`	axil4_slave_rd	Clock-gated read slave
`axil4_slave_wr_cg`	axil4_slave_wr	Clock-gated write slave
`axil4_master_r`	axil4_master_rd_mon	Clock-gated read master +

Module	Base Module	Description
d_mon_cg		monitor
axil4_master_wr_mon_cg	axil4_master_wr_mon	Clock-gated write master + monitor
axil4_slave_rd_mon_cg	axil4_slave_rd_mon	Clock-gated read slave + monitor
axil4_slave_wr_mon_cg	axil4_slave_wr_mon	Clock-gated write slave + monitor

The four *_mon_cg modules live in rtl/amba/monitor/; the four plain *_cg modules live in rtl/amba/axil4/.

2.1.2 Key Features

- **Dynamic Clock Gating:** Automatic clock disable during idle
- **Configurable Idle Threshold:** Programmable idle count before gating
- **Full Functional Equivalence:** Identical to base modules
- **Status Monitoring:** cg_gating and cg_idle status outputs
- **Runtime Bypass:** cfg_cg_enable = 0 disables gating entirely

There is **no scan/test-mode bypass port**. clock_gate_ctrl exposes only cfg_cg_enable; a DFT flow must drive that low (or bypass the ICG cell in the scan-insertion step) to hold the clock free-running during scan.

2.2 Additional Parameters (All _cg Modules)

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2^N-1 cycles)

All other parameters identical to base module.

2.3 Additional Ports (All _cg Modules)

2.3.1 Clock Gating Configuration

Port	Direction	Width	Description
cfg_cg_en	Input	1	Enable clock gating

Port	Direction	Width	Description
able			(0=disabled, 1=enabled)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating

2.3.2 Clock Gating Status

Port	Direction	Width	Description
cg_gating	Output	1	Clock currently gated (1=gated, 0=running)
cg_idle	Output	1	No activity was seen on the previous cycle (registered ~wakeup)

All other ports are identical to the base module, with one exception, and it differs by wrapper kind. On the plain _cg wrappers (axil4_master_rd_cg and the other three) the base module's busy output is **not** brought out – it is consumed internally as a wakeup term, so use cg_idle or cg_gating for idle detection there. The four _mon_cg wrappers DO expose busy; what they drop instead is debug_block_ready.

There is no cumulative gated-cycle counter on these modules. If you need one, count cg_gating in the surrounding logic on the ungated clock.

2.4 Usage Examples

```
axil4_master_rd_cg #(
    // Base parameters
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH(4)  // Up to 15 idle cycles
) u_axil_rd_cg (
    .aclk(axi_clk),
    .aresetn(axi_resetn),

    // Clock gating configuration
    .cfg_cg_enable(1'b1),          // Enable gating
    .cfg_cg_idle_count(4'd5),      // Gate after 5 idle cycles
```

```
    // AXIL signals (identical to base module)
    .fub_araddr(cpu_araddr),
    // ... rest of AR/R signals ...

    // Clock gating status
    .cg_gating(rd_clk_gated),
    .cg_idle(rd_idle)
);
```

2.5 Clock Gating Behavior

The `_cg` wrapper builds a wakeup term from the module's channel activity and the base module's internal `busy`, and hands it to `amba_clock_gate_ctrl`, which registers it and drives `clock_gate_ctrl` and its ICG cell. For `axil4_master_rd_cg` the terms are:

```
assign user_valid = fub_arvalid || fub_rvalid || int_busy;
assign axi_valid  = m_axil_arvalid || m_axil_rvalid;
```

> This term used to read `fub_rready`. A peer's READY must never appear in the activity term: a consumer that parks its response-ready high while idle is behaving correctly, and folding that in pins the block permanently awake and defeats gating entirely – silently, since function is unaffected. Ten `_cg` wrappers carried that defect until 2026-09-02.

Gating Conditions (All Must Be True): 1. No activity in `user_valid` or `axi_valid` 2. Idle for `cfg_cg_idle_count` consecutive cycles (the idle counter counts down to 0 and holds) 3. Gating enabled (`cfg_cg_enable = 1`)

Ungating Conditions (Any Triggers Ungating): 1. Any `*valid` signal asserted on either side of the module 2. `fub_rvalid` asserted (this module presenting read data), or the base module still `busy`

Gating latency: `cfg_cg_idle_count + 1` clocks after the internal wakeup deasserts, which is `cfg_cg_idle_count + 2` clocks after the last bus activity on AXI4-Lite (one extra for the `r_wakeup` flop).

Ungating latency: activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. The AXIL4 `_cg` wrappers drive `user_valid`/`axi_valid` combinationally, so there is exactly **1 register stage** — `r_wakeup` inside `amba_clock_gate_ctrl` — and the first gated-clock rising edge available to the block arrives **2 clocks** after activity asserts, not 0. Note that `clock_gate_ctrl` contributes no flop of its own: `w_gate_enable` is combinational from `wakeup` straight into the ICG enable, so the second clock is the released edge itself rather than a further register stage. The wrapper covers this by forcing the module's `ready` outputs low while `cg_gating` is asserted, for example:

```
assign fub_aready   = cg_gating ? 1'b0 : int_aready;
assign m_axil_rready = cg_gating ? 1'b0 : int_rready;
```

so a request presented while gated is simply not accepted until the clock is running again. No data is lost, but a transaction that arrives into a gated interface pays the wakeup cost before its first handshake.

2.6 Configuration Guidelines

2.6.1 Idle Count Selection

Aggressive Gating (Register Access):

```
.cfg_cg_idle_count(4'd1)   // Gate after 1 idle cycle
// AXI4-Lite typical: Burst register access with idle gaps
```

Moderate Gating (Balanced):

```
.cfg_cg_idle_count(4'd5)   // Gate after 5 idle cycles
// General purpose configuration
```

Conservative Gating (Low Latency):

```
.cfg_cg_idle_count(4'd10)  // Gate after 10 idle cycles
// Critical control paths, minimal overhead
```

Disable Gating:

```
.cfg_cg_enable(1'b0)       // Clock gating disabled
// 100% utilization or ultra-low-latency paths
```

2.6.2 When to Use Clock Gating

Recommended for: - Register access patterns (idle between accesses) - Low-duty-cycle control interfaces - Power-constrained systems

Not recommended for: - 100% utilization scenarios - Ultra-low-latency critical paths - Continuous register polling

---

2.7 Power Savings Estimates

2.7.1 Typical Savings (AXI4-Lite Register Access)

The percentages below are **rough expectations for dynamic clock power in the gated block only**, relative to the same module with `cfg_cg_enable = 0` at the same frequency. They are not

measured power-analysis results, they exclude leakage (which gating does not reduce), and they exclude the clock tree upstream of the ICG cell. Run power analysis on your own target before relying on them.

Traffic Pattern	Duty Cycle	Idle Count	Est. Dynamic Power Savings
Burst register reads	20%	1	35-40%
Sporadic writes	10%	1	40-45%
Periodic polling	50%	5	20-25%
High activity	80%	5	5-10%

Note: AXI4-Lite often has better power savings than AXI4 due to: - Single-beat transactions (more idle gaps) - Simpler control (faster idle detection) - Register access patterns (bursty by nature)

2.8 Complete Documentation

For full clock gating details, see: - **AXI4 Clock Gating Guide** - Complete reference with: - Detailed gating behavior and state machine - Power savings calculations and examples - Dynamic configuration patterns - Synthesis considerations

AXIL4-specific notes: - Same architecture as AXI4 clock gating - Typically better power savings (bursty register access) - Simpler configuration (fewer outstanding transactions)

2.9 Related Documentation

2.9.1 Base Modules

- **axil4_master_rd** - Base read master
- **axil4_master_wr** - Base write master
- **axil4_slave_rd** - Base read slave
- **axil4_slave_wr** - Base write slave
- **Monitor modules** - Base monitor modules

2.9.2 Architecture

- **AXI4 Clock Gating Guide** - Complete reference
- **rtl-amba Overview** - Complete AMBA subsystem
- **AXIL4 Index** - AXIL4 module index

Last Updated: 2026-07-19

2.10 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

3 AXIL4 Master Read

Module: `axil4_master_rd.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

3.1 Overview

The AXIL4 Master Read module gives a master device a buffered AXI4-Lite read interface. AXI4-Lite is the simplified subset of AXI4 designed for control-register access: reduced signal count, single-beat transactions only. Think of it as a clean pipe — your frontend logic and the interconnect each get their own valid/ready pair, and neither ever sees a combinational path to the other.

3.1.1 Key Features

- **AXI4-Lite Protocol:** Simplified read-only interface (AR and R channels)
- **Single-Beat Transactions:** No burst support (always 1 transfer)
- **Buffered Channels:** Configurable skid buffers for timing closure
- **Elastic Buffering:** Decouples frontend and backend timing
- **Activity Monitoring:** Busy signal for clock gating
- **Minimal Latency:** 1-cycle buffer overhead per channel

3.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	Address bus width (typically 32)

Parameter	Type	Default	Description
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AR	int	2	AR channel skid buffer depth, in entries
SKID_DEPTH_R	int	4	R channel skid buffer depth, in entries

3.2.1 Derived Parameters

Each channel's skid buffer stores the whole channel payload as one packed vector, so the derived *Size parameters are just the sum of the widths of the signals concatenated into that buffer.

Parameter	Calculation	Description
AW	AXIL_ADDR_WIDTH	Internal address width alias
DW	AXIL_DATA_WIDTH	Internal data width alias
ARSize	AW+3	AR packet size: {araddr[AW-1:0], arprot[2:0]} — the 3 is ARPROT
RSize	DW+2	R packet size: {rdata[DW-1:0], rresp[1:0]} — the 2 is RRESP

3.3 Ports

3.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	Active-low asynchronous reset

3.3.2 Frontend AXI4-Lite Read Interface (Input Side)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——-|———-| | `fub_araddr` | Input | AW | Read address | | `fub_arprot` | Input | 3 | Protection attributes | | `fub_arvalid` | Input | 1 | Address valid | | `fub_arready` | Output | 1 | Address ready (from buffer) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——-|———-| | `fub_rdata` | Output | DW | Read data | | `fub_rresp` | Output | 2 | Read response (OKAY, SLVERR, DECERR) | | `fub_rvalid` | Output | 1 | Data valid (from buffer) | | `fub_rready` | Input | 1 | Data ready |

3.3.3 Backend AXI4-Lite Master Interface (Output Side)

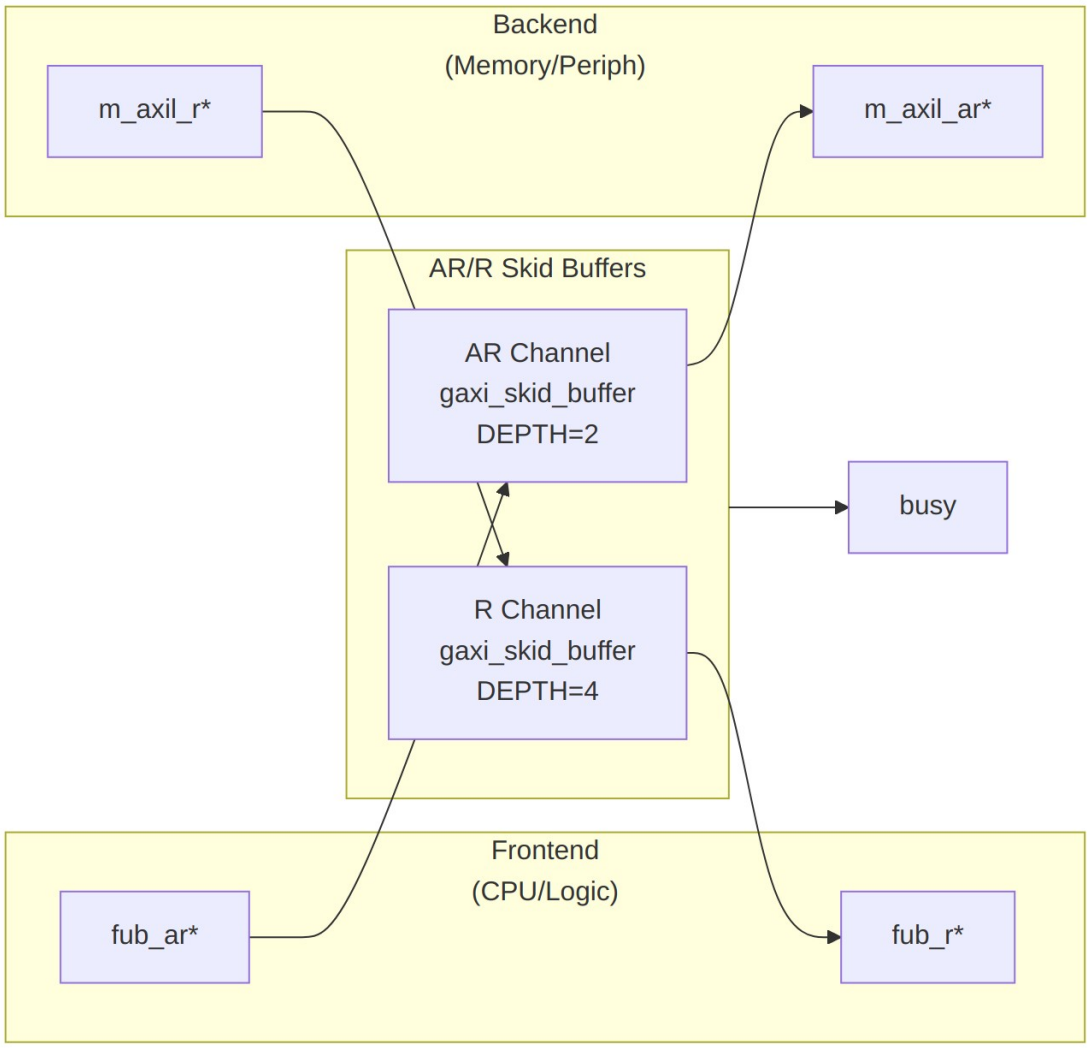
Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——-|———-| | `m_axil_araddr` | Output | AW | Read address (to slave) | | `m_axil_arprot` | Output | 3 | Protection attributes | | `m_axil_arvalid` | Output | 1 | Address valid (from buffer) | | `m_axil_arready` | Input | 1 | Address ready (from slave) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——-|———-| | `m_axil_rdata` | Input | DW | Read data (from slave) | | `m_axil_rresp` | Input | 2 | Read response | | `m_axil_rvalid` | Input | 1 | Data valid (from slave) | | `m_axil_rready` | Output | 1 | Data ready (to slave) |

3.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Interface active (for clock gating)

3.4 Functional Description



Module Architecture

3.4.1 Read Transaction Sequence

One address in, one data beat out — each channel pays one clock in its skid buffer along the way:

```
Cycle 1:  fub_araddr=0x1000, fub_arvalid=1, fub_arprot=0
          fub_arready=1 (buffer accepts)

Cycle 2:  AR buffer forwards to backend:
          m_axil_araddr=0x1000, m_axil_arvalid=1
          m_axil_arready=1 (slave accepts)

Cycle 3-5: Slave processes read
```

```
Cycle 6:  m_axil_rdata=0xDEADBEEF, m_axil_rresp=OKAY
          m_axil_rvalid=1, m_axil_rready=1 (R buffer accepts)

Cycle 7:  R buffer forwards to frontend:
          fub_rdata=0xDEADBEEF, fub_rresp=OKAY
          fub_rvalid=1, fub_rready=1 (frontend accepts)
```

3.5 Timing Characteristics

3.5.1 Latency

Path	Cycles	Notes
Frontend → Backend (AR)	1	Skid buffer overhead
Backend → Frontend (R)	1	Skid buffer overhead
Total read latency	Slave latency + 2	AR + Slave + R

3.5.2 Throughput

Scenario	Rate	Notes
Back-to-back reads	1 read/cycle	If buffers not stalled
Typical peripheral	1 read/N cycles	N = slave response time

3.5.3 Resource Usage

These are **order-of-magnitude estimates**, not measured synthesis results — no target device, tool version, or optimization setting is attached to them. They scale with `AXIL_DATA_WIDTH` and the `SKID_DEPTH_*` settings. Synthesize for your own device before budgeting area.

Resource	Count	Notes
LUTs	~200	Estimate, 32-bit data, default depths
FFs	~150	Buffer state + control
BRAM	0	Skid buffers are flop-based

3.5.4 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

3.6 Usage Examples

3.6.1 Basic Integration

```
axil4_master_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),      // 2 entries
    .SKID_DEPTH_R(4)        // 4 entries
) u_axil_master_rd (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),

    // Frontend interface (from CPU)
    .fub_araddr     (cpu_araddr),
    .fub_arprot     (cpu_arprot),
    .fub_arvalid    (cpu_arvalid),
    .fub_arready    (cpu_arready),

    .fub_rdata      (cpu_rdata),
    .fub_rresp      (cpu_rresp),
    .fub_rvalid     (cpu_rvalid),
    .fub_rready     (cpu_rready),

    // Backend interface (to peripheral/memory)
    .m_axil_araddr  (periph_araddr),
    .m_axil_arprot  (periph_arprot),
    .m_axil_arvalid (periph_arvalid),
    .m_axil_arready (periph_arready),
```

```
    .m_axil_rdata   (periph_rdata),
    .m_axil_rresp   (periph_rresp),
    .m_axil_rvalid  (periph_rvalid),
    .m_axil_rready  (periph_rready),

    // Status
    .busy           (rd_busy)
);
```

3.6.2 Register Read Example (Testbench Stimulus)

The snippet below is **testbench stimulus, not RTL**. It uses an `initial` block and blocking assignments to drive the frontend ports and is not synthesizable. In a real design the frontend handshake is driven by an `always_ff` block or by the register-access state machine of the requesting logic.

```
// Read from control register at 0x1000
initial begin
    @(posedge aclk);
    fub_araddr  = 32'h1000;
    fub_arprot  = 3'b000;  // Unprivileged, secure, data access
    fub_arvalid = 1'b1;

    @(posedge aclk);
    while (!fub_arready) @(posedge aclk);  // Wait for buffer accept
    fub_arvalid = 1'b0;

    // Wait for read data
    fub_rready = 1'b1;
    @(posedge aclk);
    while (!fub_rvalid) @(posedge aclk);

    $display("Read data: 0x%08x, RRESP: %0d", fub_rdata, fub_rresp);
end
```

3.7 Design Notes

3.7.1 AXI4-Lite Protocol Constraints

Single-Beat Only: - No burst support (implicitly ARLEN=0, ARSIZE=log2(DW/8)) - Each AR transaction results in exactly 1 R transaction - No RLAST signal (always last)

No Out-of-Order: - No ARID/RID signals - Responses always in order - Simpler transaction tracking

Reduced Signals: - No ARCACHE, ARQOS, ARREGION, ARLOCK, ARSIZE, ARLEN, ARBURST - Only ARADDR and ARPROT for control

3.7.2 Buffer Depth Selection

`SKID_DEPTH_AR` and `SKID_DEPTH_R` are passed straight through to the `DEPTH` parameter of `gaxi_skid_buffer`. `DEPTH` is the **literal entry count**, not an exponent, and is constrained to one of 2..8 inclusive.

AR Channel (Addresses): - **SKID_DEPTH_AR** = **2** (2 entries): Typical for control registers - Increase if many back-to-back reads with slow slave response

R Channel (Data): - **SKID_DEPTH_R** = **4** (4 entries): Accommodates pipeline latency - Increase for high-latency backends (DRAM, off-chip peripherals)

3.7.3 Busy Signal

`busy` is the OR of two persistent terms and two transient terms:

Term	Kind	Condition
AR buffer occupancy	Persistent	`int_ar_count > 0`
R buffer occupancy	Persistent	`int_r_count > 0`
Frontend presenting a new address	Transient	`fub_arvalid`
Backend presenting new data	Transient	`m_axil_rvalid`

The two transient terms can be high for a single cycle, so `busy` itself can pulse. This is intentional: the clock-gate controller does not gate on `busy` directly. `amba_clock_gate_ctrl` registers activity into a `wakeup` flop and then requires `cfg_cg_idle_count` consecutive idle cycles before gating, so a one-cycle handshake reloads the idle counter rather than causing the gate to oscillate. Consumers of `busy` outside the clock-gating path should qualify or stretch it themselves if they need a level signal.

Use for: - Clock gating control (see `axil4_master_rd_cg`) - Power management - Interface idle detection

3.7.4 PROT Encoding

ARPROT[2:0]	Privilege	Security	Type
3’b000	Unprivileged	Secure	Data
3’b001	Privileged	Secure	Data
3’b010	Unprivileged	Non-secure	Data
3’b011	Privileged	Non-secure	Data

ARPROT[2:0]	Privilege	Security	Type
3'b100	Unprivileged	Secure	Instruction
3'b101	Privileged	Secure	Instruction
3'b110	Unprivileged	Non-secure	Instruction
3'b111	Privileged	Non-secure	Instruction

Per AMBA AXI (IHI 0022, A4.7): `AxPROT[0]` is privilege (1 = privileged), `AxPROT[1]` is security (1 = NON-secure), `AxPROT[2]` is type (1 = instruction). This table previously had bits 0 and 2 swapped, which only 3'b000 and 3'b111 agree on – every other row named the wrong access.

Typical: `3'b000` (unprivileged secure data) for most peripherals

---

3.8 Related Modules

3.8.1 Core Modules

- **axil4_master_wr** - Write master counterpart
- **axil4_slave_rd** - Slave read interface
- **axil4_slave_wr** - Slave write interface

3.8.2 Monitor Modules

- **axil4_master_rd_mon** - Master read with monitoring (`rtl/amba/monitor/`)

3.8.3 Clock-Gated Variants

- **axil4_master_rd_cg** - Clock-gated version

3.8.4 Supporting Infrastructure

- **gaxi_skid_buffer** - Elastic buffering (rtl/amba/gaxi/)

3.9 References

3.9.1 Specifications

- ARM IHI 0022E: AMBA AXI and ACE Protocol Specification
- Chapter 4: AXI4-Lite (simplified subset)

3.9.2 Documentation

- **rtl-amba Overview** - Complete AMBA subsystem

- **AXI4 Master Read** - Full AXI4 comparison

3.9.3 Source Code

- RTL: `rtl/amba/axil4/axil4_master_rd.sv`
- Tests: `val/amba/test_axil4_master_rd.py`
- Framework: `bin/TBClasses/components/axil4/`

Last Updated: 2026-07-19

3.10 Testing

`val/amba/test_axil4_master_rd.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_master_rd.py -v
```

3.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

4 AXIL4 Master Read Interface (Clock-Gated)

Module: `axil4_master_rd_cg.sv` **Base Module:** axil4_master_rd **Location:** `rtl/amba/axil4/` **Status:** Production Ready

---

4.1 Overview

This is the **clock-gated variant** of axil4_master_rd.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **AXIL4 Clock-Gated Variants Guide** (AXIL4-specific)

→ **Clock-Gated Variants Guide** (cross-protocol overview)

What the wrapper adds over the base module:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the AXIL4 Clock-Gated Variants Guide for the estimate table and its caveats
- **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
- **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module

---

4.2 Parameters

In addition to all axil4_master_rd parameters:

The `_cg` wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no per-domain `CG_GATE_*` parameters on this module.

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle counter; max idle count = 2^N - 1

4.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AW`	`AXIL_ADDR_WIDTH`
`DW`	`AXIL_DATA_WIDTH`

4.3 Ports

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating (0 = clock always running)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating
cg_gating	Output	1	Clock currently gated
cg_idle	Output	1	No activity on the previous cycle

The base module's busy output is **not** exposed on the _cg wrapper; it is consumed internally as a wakeup term. All other ports are identical to axil4_master_rd.

4.4 Usage Examples

```
axil4_master_rd_cg #(
    // Base module parameters (see axil4_master_rd.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime, not parameters)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd5),

    // ... all other ports same as axil4_master_rd, except `busy`,
    // which is replaced by cg_gating / cg_idle
    .cg_gating(clk_gated),
    .cg_idle(if_idle)
);
```

4.5 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The transport datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to `axil4_master_rd` and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (upstream valids plus the base module's `busy`) and `axi_valid` (downstream valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `! cg_gating`, so an upstream master sees no acceptance until the clock is running again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

4.6 Timing Characteristics

4.6.1 Buffer Depths and Latency

Parameter	Default	Channel
`SKID_DEPTH_AR`	2 entries	Skid depth on the AR channel
`SKID_DEPTH_R`	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

4.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. This wrapper wakes

on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; `cfg_monitor_enable` and the datapath are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the counter, plus one for the `r_wakeup` flop. Budget the idle count against your traffic's inter-burst gap; too small and the block wakes constantly, too large and it never gates.

4.8 Related Modules

- **axil4_master_rd** - Base module functionality
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

4.9 References

- **axil4_clock_gating_guide.md** - AXIL4 Clock Gating Guide
- **clock_gated_variants.md** - Cross-Protocol Clock Gating Guide

---

Last Updated: 2026-07-19

4.10 Testing

`val/amba/test_axil4_master_rd_cg.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 1 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_master_rd_cg.py -v
```

`val/amba/test_cg_peer_ready.py` additionally asserts that this wrapper gates with the peer's READY parked high – the property the activity term exists to satisfy. See `vault/handbook/design/clock-gating-activity-terms.md`.

4.11 Navigation

- ← Back to AXIL4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

5 AXIL4 Master Read with Monitoring

Module: `axil4_master_rd_mon.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

5.1 Overview

Combines **axil4_master_rd** with the core **axi_monitor_filtered** for transaction monitoring. Simplified for AXI4-Lite (single-beat, no burst, fixed ID=0).

5.1.1 Key Features

- All features of base **axil4_master_rd** module
- **Integrated Monitoring:** Uses shared axi_monitor_filtered (rtl/amba/monitor/)
- **2-Level Filtering:** packet-type masks, then per-event-code masks. `err_select` is RESERVED – it feeds only the conflict check and routes nothing.
- **Error Detection:** SLVERR/DECERR, timeouts, orphaned read data (protocol-violation events are write-monitor only)
- **128-bit Monitor Bus:** Standardized packet format paired with 64-bit side-band timestamp
- **Reduced Complexity:** MAX_TRANSACTIONS=8 (vs 16-32 for AXI4)

5.2 Parameters

5.2.1 Additional Parameters

Beyond the base module's parameters:

Parameter	Type	Default	Description
UNIT_ID	logic [7:0]	8'h01	8-bit unit identifier emitted in the unit_id packet field
AGENT_ID	logic [15:0]	16'h000A	16-bit agent identifier emitted in the agent_id packet field
MAX_TRANSACTIONS	int	8	Max outstanding transactions. Reduced for AXI4-Lite; the AXI4 wrappers default to 16.
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when cfg_threshold_enable=1. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N

Parameter	Type	Default	Description
			independent [low, high] ranges; feeds the shared allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
ACLK_MHZ	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in counter_freq_invariant. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently.**
CFI_MIN_FREQ_MHZ / CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Bounds of the freq-invariant counter LUT (cfg_freq_sel indexes within them). Equal bounds means one entry and cfg_freq_sel has no effect.
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue, forwarded to axi_monitor_base.
NUM_BANKS	int	1	Banked transaction tables. >**1 on a WRITE monitor requires USE_WDATA_ORDER_Q=1** – axi_monitor_trans_mgr fails elaboration otherwise.
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice

Parameter	Type	Default	Description
			filter, inherited from the shared monitor core. **Leave at 0 on AXI4-Lite.** The wrapper hardwires `cmd_id`/`data_id`/`resp_id` to `1'b0`, so enabling this with `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring.
`ID_MATCH_BASE`	int	`0`	First ID this instance owns when `ID_FILTER_ENABLE=1`. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
`ID_MATCH_COUNT`	bit	`0`	Number of IDs this instance owns from `ID_MATCH_BASE`. 0 = all IDs.

5.2.2 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. These are forwarded to `axi_monitor_base`; by default the classic cones are on and the debug cone is off.

Parameter	Type	Default	Effect when 0
`ENABLE_ERROR_LOGIC`	bit	1	Drop the error-detection cone
`ENABLE_TIMEOUT_LOGIC`	bit	1	Drop the timeout cone **and** the `axi_monitor_timeout` instance
`ENABLE_COMPL_LOGIC`	bit	1	Drop the completion cone
`ENABLE_THRESHOLD_LOGIC`	bit	1	Drop the threshold cone

Parameter	Type	Default	Effect when 0
ENABLE_PERF_LOGIC	bit	1	Gates only the reporter's legacy perf-packet cone and the two lifetime counters – the window FSM and meters are unconditional
ENABLE_DEBUG_LOGIC	bit	0	(off by default) drop the debug cone

The former CAM_PIPELINE / TRANS_CAM_PIPELINE parameters were removed — the transaction CAM is now always pipelined. Inside this wrapper ENABLE_PERF_PACKETS is tied to 1'b1 (perf packet path always instantiated, gated by ENABLE_PERF_LOGIC) and ENABLE_DEBUG_MODULE is tied to 1'b0.

5.2.3 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

5.3 Ports

5.3.1 Monitor Configuration

Port	Direction	Width	Description
cfg_monitor_enable	Input	1	Master runtime gate. 0 = monitor inert: no allocation, transaction CAM held clear, and the upstream ready is never stalled (a disabled monitor cannot block the

Port	Direction	Width	Description
			datapath). 1 = normal operation
cam_clear	Input	1	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
cfg_error_enable	Input	1	Enable error packets
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_compl_enable	Input	1	Enable transaction-completion packets
cfg_threshold_enable	Input	1	Enable threshold-crossed packets
cfg_perf_enable	Input	1	Enable performance packets
cfg_debug_enable	Input	1	Enable debug/trace packets (the 6th “debug cone” reporter sub-block)
cfg_timeout_cycles	Input	16	Unified coarse timeout control: 0 = legacy full-scale (15 ticks), 1-15 = that many timer ticks per phase, >15 saturates at 15. One value drives all three phase counts (addr/data/resp), measured in cfg_freq_sel-scaled timer ticks, not raw clock cycles
cfg_latency_threshold	Input	32	Latency alert threshold

The debug cone is the 6th reporter sub-block (`axi_monitor_reporter_debug`), enabled by `cfg_debug_enable` and synthesized only when `ENABLE_DEBUG_LOGIC = 1`. The related `cfg_debug_level` (4b) and `cfg_debug_mask` (16b) inputs of `axi_monitor_base` are **not** exposed on this AXIL wrapper — they are tied to `4'h0` / `16'h0` internally; `cfg_active_trans_threshold` is driven from the `ACTIVE_TRANS_THRESHOLD` parameter (default `MAX_TRANSACTIONS/2`).

5.3.2 Performance Monitoring Ports

Port	Direction	Width	Description
`cfg_start_event_sel`	Input	3	Window **start** event source select
`cfg_end_event_sel`	Input	3	Window **end** event source select
`cfg_start_trigger`	Input	1	Pulse: open the measurement window
`cfg_end_trigger`	Input	1	Pulse: close the measurement window
`cfg_window_force_close`	Input	1	Software override: force the window closed
`window_active`	Output	1	High while a measurement window is open
`window_cycles`	Output	32	Cycles elapsed in the current window
`perf_prod_cycles`	Output	32	valid && ready cycles
`perf_bp_cycles`	Output	32	valid && !ready cycles (back-pressure)
`perf_starv_cycles`	Output	32	!valid && ready cycles (starvation)
`perf_idle_cycles`	Output	32	!valid && !ready cycles
`perf_beat_count`	Output	32	data beats transferred
`perf_byte`	Output	64	bytes transferred

Port	Direction	Width	Description
_count			
perf_burst_count	Output	32	AR (address-phase) handshakes

When perfmon is unused, the integrating block ties `cfg_start_event_sel`/`cfg_end_event_sel` to `3'b111` and the remaining `cfg_*` perfmon inputs to 0. When `USE_MONITOR = 0` all perfmon outputs are tied to 0.

5.3.3 Filtering Masks (9 masks total)

All nine are 16-bit inputs. `cfg_axi_pkt_mask` is indexed by the 4-bit `packet_type` field; the per-type event masks are indexed by the low nibble of `event_code`. In every case a **set bit drops** the matching packet.

Port	Width	Description
cfg_axi_pkt_mask	16	Level 1: drop mask indexed by `packet_type`
cfg_axi_err_select	16	RESERVED – routes nothing. `axi_monitor_filtered` uses it in exactly one place, `cfg_conflict_error = |(cfg_axi_pkt_mask & cfg_axi_err_select)`, so its only effect is to raise that flag on an overlap.
cfg_axi_error_mask	16	Level 3: mask individual error events
cfg_axi_timeout_mask	16	Level 3: mask individual timeout events
cfg_axi_compl_mask	16	Level 3: mask individual completion events
cfg_axi_thresh_mask	16	Level 3: mask individual threshold events
cfg_axi_perf_mask	16	Level 3: mask individual performance events
cfg_axi_addr_mask	16	Level 3: mask individual address-match events

Port	Width	Description
cfg_axi_debug_mask	16	Level 3: mask individual debug events

There is no cfg_axi_full_mask port.

5.3.4 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor packet valid
monbus_ready	Input	1	Downstream ready
monbus_packet	Output	128	monitor_packet_t (128-bit format)
monbus_timestamp	Output	64	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	Input	64	Free-running counter from monbus_group_core, sampled at packet emission

5.3.5 Status

Port	Direction	Width	Description
busy	Output	1	Interface active
active_transactions	Output	8	Current outstanding count
error_count	Output	16	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when ENABLE_PERF_LOGIC=0 or USE_MONITOR=0)
transaction_count	Output	32	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter

Port	Direction	Width	Description
			perf counter; reads 0 when ENABLE_PERF_LOGIC=0 or USE_MONITOR=0)
cfg_conflict_error	Output	1	Set when cfg_axi_pkt_mask and cfg_axi_err_select overlap

5.4 Functional Description

5.4.1 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of R-channel (read-data) utilization counters to axi_monitor_base. All counters accumulate **only while a window is open** (window_active = 1) and hold their values between windows so the host can read a completed window's totals. cfg_perf_enable does NOT gate them: it selects the window start/end event (it is edge-detected for cfg_start_event_sel modes 010/011) and enables the perf PACKET class. The counters themselves advance whenever a window is open, enabled or not. ENABLE_PERF_LOGIC = 0 does NOT drop them: the window FSM and its counters are unconditional always_ff blocks in axi_monitor_base, outside every generate. That parameter gates only g_perf in the reporter – the legacy perf-packet cone and the two lifetime counters. USE_MONITOR = 0 is what ties the perfmon outputs off.

> Avoid enabling completion (cfg_compl_enable) and performance (cfg_perf_enable) packets simultaneously under heavy traffic — the monitor bus sustains at most one packet per two cycles. Runtime-disabling either class is safe (terminal entries auto-retire; see axi_monitor_reporter); alternatively, cfg_axi_pkt_mask drops the packets while keeping marking and counting. See docs/user-guides/AXI_Monitor_Configuration_Guide.md.

5.4.1.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- cfg_start_event_sel / cfg_end_event_sel (3-bit) select the event source (e.g. 3'b010 selects the cfg_perf_enable edge).
- cfg_start_trigger / cfg_end_trigger pulses fire the window directly from an engine or CSR.

- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

5.4.1.2 Utilization Counters (R channel)

Every in-window cycle is classified by the R-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	rvalid && rready	productive beat transferred
`perf_bp_cycles`	32	rvalid && !rready	back-pressure (data offered, consumer not ready)
`perf_starv_cycles`	32	!rvalid && rready	starvation (consumer ready, no valid data)
`perf_idle_cycles`	32	!rvalid && !rready	idle

The four buckets sum to `window_cycles`, so utilization = `perf_prod_cycles / window_cycles`.

5.4.1.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats × (1 << AxSIZE); AXIL is fixed at `AxSIZE=3'b010` (4 bytes)
`perf_burst_count`	32	AR (read-address) handshakes

AXI4-Lite note: every transaction is a single data beat (ARLEN is implicitly 0), so `perf_burst_count` counts AR handshakes = transactions and `perf_beat_count` equals the transaction count. Average burst length (`perf_beat_count / perf_burst_count`) is therefore always 1.

5.4.2 Monitor Backpressure (block_ready)

`block_ready` is a flow-control net inside the wrapper, and it IS brought out: `debug_block_ready` is an output port of this module (the `_cg` wrapper ties it off, so use the base module when you need the tap). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `fub_axil_aready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `fub_axil_aready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property, `ap_disabled_never_stalls`; this module has no `ifdef FORMAL` block of its own, so here the guarantee rests on the gate expression above, not a proof).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `monitor_common_pkg`), and `block_ready` re-asserts at occupancy `< MAX_TRANSACTIONS - (reserve - 1)` – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

5.4.3 Address-Range Checker

With `N_ADDR_RANGES > 0` the wrapper instantiates the shared allowlist checker (`axi_monitor_addr_check`). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a `PktTypeAddrMatch` (`4'h8`) packet with event code `AXI_ADDR_RANGE_MATCH` (`8'h01`), gated by `cfg_debug_enable`.

- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a `PktTypeError` (`4'h0`) packet with event code `AXI_ERR_ADDR_RANGE` (`8'h0D`), gated by `cfg_error_enable`.

This wrapper does not expose `ADDR_RANGE_IS_ERROR`; all ranges default to the DEBUG (match) flavor.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive bounds.

Event encoding: `event_data[63:60]` = `range_index` (matching DEBUG range, or `4'hF` sentinel on an ERROR miss); `event_data[59:0]` = full address. **Filtering:** AddrMatch is dropped by `cfg_axi_addr_mask[1]`, the ADDR_RANGE error by `cfg_axi_error_mask[13]`. See the checker page for coalescing + formal properties.

5.4.4 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

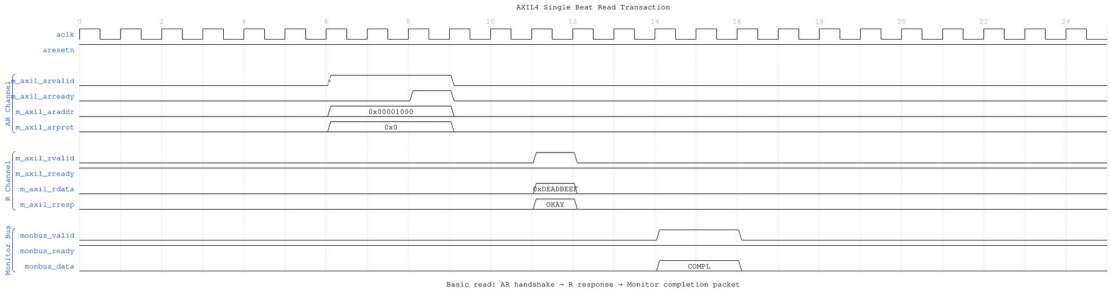
Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is what makes it safe to reprogram live. Contrast the address-range CHECKER above, which re-evaluates per command.

There is no runtime ID filter here. AXI-Lite has no transaction IDs, so this wrapper ties `cfg_id_filter_enable` / `cfg_id_match_base` / `cfg_id_match_count` off on the inner monitor rather than exposing them — a filter keyed on a field the protocol lacks has nothing to match against.

5.5 Waveforms

5.5.1 Scenario 1: Single-Beat Read Transaction

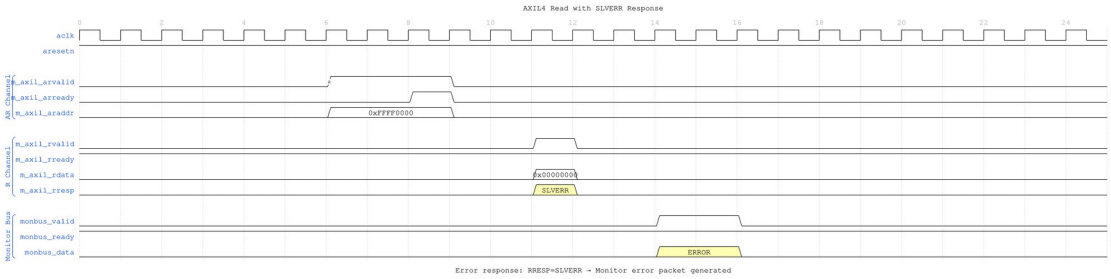


Single Beat Read

WaveJSON: single_beat_read_001.json

Key Observations: - AR channel handshake: ARVALID asserted, ARREADY responds - R channel response: Slave returns data with RRESP=OKAY - Monitor generates completion packet when R phase completes - Single-beat transaction: No burst length (implicit ARLEN=0)

5.5.2 Scenario 2: Read Error (SLVERR)

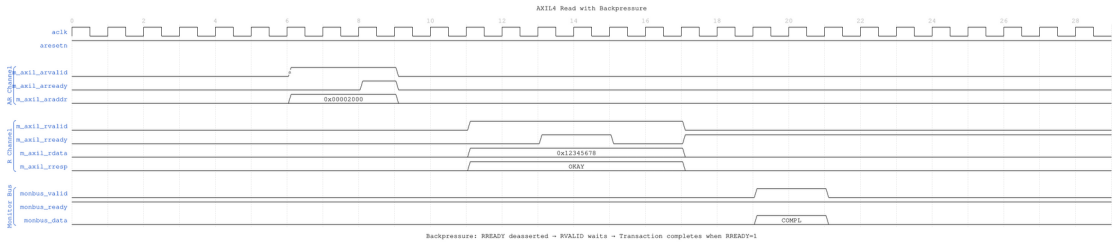


Read Error SLVERR

WaveJSON: read_error_slverr_001.json

Key Observations: - Invalid address triggers RRESP=SLVERR - Monitor detects error response and generates ERROR packet - Transaction completes despite error (data may be undefined) - Error packet includes address and response code

5.5.3 Scenario 3: Read with Backpressure



Read Backpressure

WaveJSON: read_backpressure_001.json

Key Observations: - Master not ready: RREADY deasserted - Slave holds RVALID until RREADY=1 - Monitor tracks extended latency - Completion packet generated after handshake

---

5.6 Usage Examples

```
axil4_master_rd_mon #(
    // Base parameters
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Monitor parameters
    .UNIT_ID(1),
    .AGENT_ID(10),
    .MAX_TRANSACTIONS(8),
    .ENABLE_FILTERING(1)
) u_axil_rd_mon (
    .aclk(axi_clk),
    .aresetn(axi_resetn),

    // AXIL interfaces (same as axil4_master_rd)
    .fub_axil_araddr(cpu_araddr),
    // ... other AR/R signals ...

    // Monitor configuration
    .cfg_monitor_enable(1'b1),
    .cfg_error_enable(1'b1),
    .cfg_timeout_enable(1'b1),
    .cfg_perf_enable(1'b0),      // Avoid congestion
    .cfg_timeout_cycles(16'd10),  // 10 timer ticks per phase (>15
saturates)
```

```
    .cfg_freq_sel    (4'd0),   // counter_freq_invariant LUT index;
scales the 1 us tick
    .cam_clear       (1'b0),   // hold high one cycle while idle to
clear the CAM -- do NOT leave unconnected
    .cfg_latency_threshold(32'd500),

    // Filtering masks - a SET bit DROPS that packet type
    .cfg_axi_pkt_mask(16'b1111_1111_1111_0110),  // Keep Error (bit 0)
+ Timeout (bit 3)
    // ... other masks ...

    // Monitor bus
    .monbus_valid(mon_valid),
    .monbus_ready(mon_ready),
    .monbus_packet(mon_data),
    .monbus_timestamp(mon_time)
);
```

Mask polarity. `cfg_axi_pkt_mask` is 16 bits because it is indexed by the 4-bit `packet_type` field — one bit per packet type — not by the 128-bit packet width. `axi_monitor_filtered` computes `pkt_drop = cfg_axi_pkt_mask[pkt_type]`, so a set bit **drops** that type. To pass only `PktTypeError` (4'h0) and `PktTypeTimeout` (4'h3), clear bits 0 and 3 and set the rest: `16'b1111_1111_1111_0110`. See the Monitor Packet Specification for the full packet-type enum.

5.7 Design Notes

5.7.1 AXI4-Lite Simplifications

vs Full AXI4 Monitoring: - **Fixed ID:** Always ID=0 (no out-of-order tracking) - **Single-Beat:** No burst handling (ARLEN implicitly 0) - **Reduced Tables:** MAX_TRANSACTIONS=8 (vs 16-32 for AXI4) - **Same Infrastructure:** Uses axi_monitor_filtered like AXI4

5.8 Timing Characteristics

5.8.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R

Parameter	Default	Channel
		channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

5.9 Related Modules

5.9.1 Base Module

- **axil4_master_rd** - Functional module documentation

5.9.2 Monitor Infrastructure

- **AXI4 Master Read Mon** - Full AXI4 monitoring (detailed reference)
- **axi_monitor_filtered** - Core monitor engine (`rtl/amba/monitor/`)
- **Monitor Configuration Guide** - Configuration strategies

5.9.3 Related Modules

- **axil4_master_wr_mon** - Master write with monitoring
- **axil4_slave_rd_mon** - Slave read with monitoring

Last Updated: 2026-07-19

5.10 Testing

`val/amba/test_axil4_master_rd_mon.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_master_rd_mon.py -v
```

5.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**

6 AXIL4 Master Read Monitor (Clock-Gated)

Module: `axil4_master_rd_mon_cg.sv` **Base Module:** axil4_master_rd_mon **Location:** `rtl/amba/axil4/` **Status:** Partial — see Implementation Status

---

6.1 Overview

`axil4_master_rd_mon_cg` wraps axil4_master_rd_mon and adds a power-management control and status interface. All monitoring, filtering, address-range checking, and performance-monitoring behavior is that of the base module; see axil4_master_rd_mon.md for the complete functional specification.

6.1.1 Implementation Status

This wrapper gates the monitor's clock for real. It instantiates `amba_clock_gate_ctrl`, and the base `axil4_master_rd_mon` inside it is driven from `gated_aclk`, not from `aclk`.

What it does:

1. **Gates the clock.** `amba_clock_gate_ctrl` counts `cfg_cg_idle_count` idle cycles and stops `gated_aclk`, reporting on `cg_gating` (the gated clock is stopped) and `cg_idle` (no activity observed).
2. **Holds off the interfaces while gated.** `fub_axil_arready` and `m_axil_rready` are forced low under `cg_gating`, so nothing is accepted into a stopped clock.
3. **Masks the monbus valid while gated** (`monbus_valid = w_monbus_valid && !cg_gating`), which is what makes delivery exactly-once across a gating edge rather than a held valid replayed on wake.

Two earlier versions of this page were wrong in opposite directions, so both are worth naming. One described a `cg_cycles_saved` output, a `cfg_cg_idle_threshold` input and `ENABLE_CLOCK_GATING` / `CG_IDLE_CYCLES` parameters; none of those exist anywhere in `rtl/amba`. The other – the correction to the first – concluded that the wrapper therefore gates nothing and "will not reduce dynamic power". That was the more damaging error: it is false, and it steers an integrator away from the right part. The real controls are `cfg_cg_enable` and

`cfg_cg_idle_count` (width `CG_IDLE_COUNT_WIDTH`); there is no cycles-saved counter of any kind.

Gating behaviour is asserted directly by `val/amba/test_mon_cg_gating.py` (phase 5 for the ready hold-off, phase 6 for exactly-once monbus delivery).

6.2 Parameters

In addition to all axil4_master_rd_mon parameters (including `USE_MONITOR` and `N_ADDR_RANGES`):

Parameter	Type	Default	Description
`ACLK_MHZ`	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was unreachable through this wrapper until 2026-09-01.
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	Lowest frequency the tick LUT must cover (dynamic-frequency builds).
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	Highest frequency the tick LUT must cover.
`USE_WDATA_ORDER_Q`	bit	0	Write-data ordering queue. Required (=1) whenever `NUM_BANKS` > 1.
`NUM_BANKS`	int	1	Transaction-table banking. The `USE_WDATA_ORDER_Q` pairing rule applies to

Parameter	Type	Default	Description
			WRITE monitors only – `axi_monitor_trans_mgr` guards on `(NUM_BANKS > 1) && !IS_READ && ! USE_WDATA_ORDER_Q`, so a read monitor may bank freely.
`ADDR_FILTER_ENABLE`	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves `cfg_addr_filter_enable` low filters nothing and looks broken.
`CG_IDLE_COUNT_WIDTH`	int	4	Width of `cfg_cg_idle_count`; sets the longest programmable idle threshold
`ADD_PIPELINE_STAGE`	bit	`0`	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
`ENABLE_FILTERING`	bit	`1`	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing.

There are no `CG_GATE_MONITOR`, `CG_GATE_REPORTER`, or `CG_GATE_TIMERS` parameters, and no independent gating domains. Earlier revisions of this document described such a scheme; it was never implemented.

6.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

6.3 Ports

6.3.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
cfg_addr_filter_enable	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever ADDR_FILTER_ENABLE says
cfg_addr_filter_low	Input	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	Input	ADDR_WIDTH	Window limit, inclusive

There is no runtime ID filter on AXI4-Lite: the protocol has no IDs to match.

Base-module ports are forwarded unchanged EXCEPT `debug_block_ready`, which this wrapper does not bring out (use the base module when you need that tap). That includes the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM, driven from the harness clear control bit, e.g. CTRL[4]. The wrapper adds four ports:

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Arms clock gating. It reaches amba_clock_gate_ctrl only –

Port	Direction	Width	Description
			cfg_monitor_enable is forwarded to the inner monitor untouched, so 0 here does NOT disable the monitor, it just leaves the clock free-running.
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before the clock gates. The clock stops cfg_cg_idle_count + 2 cycles after the last bus activity (one extra for the r_wakeup flop).
cg_gating	Output	1	High while the clock is gated.
cg_idle	Output	1	High while the activity terms are quiet.

The base module's busy output remains available on this wrapper.

6.4 Functional Description

6.4.1 Performance Monitoring

The wrapper forwards the base module's full performance-monitoring interface to axi_monitor_base **unchanged** — the power-management interface neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for axil4_master_rd_mon:

- **Config inputs:** cfg_perf_enable, cfg_start_event_sel, cfg_end_event_sel, cfg_start_trigger, cfg_end_trigger, cfg_window_force_close
- **Status / counters:** window_active, window_cycles, perf_prod_cycles, perf_bp_cycles, perf_starv_cycles, perf_idle_cycles, perf_beat_count, perf_byte_count, perf_burst_count

The completion/threshold/debug enables (cfg_compl_enable, cfg_threshold_enable, cfg_debug_enable) and the synthesis-cone parameters (ENABLE_ERROR_LOGIC, ENABLE_TIMEOUT_LOGIC, ENABLE_COMPL_LOGIC, ENABLE_THRESHOLD_LOGIC, ENABLE_PERF_LOGIC, ENABLE_DEBUG_LOGIC) are likewise forwarded unchanged. The utilization

buckets watch the **R** (read-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AR handshakes = transactions.

> Note that `cfg_cg_enable = 0` does NOT disable the monitor. It only disarms clock gating, leaving the inner monitor on a free-running clock and behaving exactly like the base module. Use `cfg_monitor_enable = 0` to actually stop monitoring.

6.5 Usage Examples

```
axil4_master_rd_mon_cg #(
    // Base module parameters (see axil4_master_rd_mon.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Monitor parameters
    .UNIT_ID(8'h01),
    .AGENT_ID(16'h000A),
    .MAX_TRANSACTIONS(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Power-management interface
    .cfg_cg_enable(1'b1),           // arm gating; 0 = clock free-
runs
    .cfg_cg_idle_count(4'd4),       // idle cycles before the clock
stops
    .cg_gating(cg_gating),          // high while the gated clock is
stopped
    .cg_idle(cg_idle),

    // ... all other ports same as axil4_master_rd_mon
);
```

6.6 Design Notes

Monitor cost is not incremental. The transaction table is `bus_transaction_t` x `MAX_TRANSACTIONS`, and the reporter keeps a second full copy (`r_trans_table_local`), so a

monitored interface is a multiple of the unmonitored one rather than a few percent on top. `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`perf_byte_count` scales with the bus width. `cmd_size` is derived as `$clog2(AXIL_DATA_WIDTH/8)`. It was hardwired to `3'b010` (4 bytes) until 2026-09-02, which halved every byte count on a 64-bit Lite bus without failing anything. `val/amba/test_axil_perf_byte_count.py` pins it at both legal widths.

Do not enable completion and performance packets together under load. The monitor bus sustains at most one packet per two cycles. Use `cfg_axi_pkt_mask` to drop a class while keeping its marking and counting.

The ID filter is inert and must stay that way. AXI4-Lite has no IDs, so this wrapper ties the monitor's ID inputs to zero; enabling `ID_FILTER_ENABLE` with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring rather than narrowing it.

6.7 Testing

A clock IS gated here – that is the wrapper's entire purpose. `amba_clock_gate_ctrl` drives the inner monitor's clock, and it stops when the activity terms go quiet for `cfg_cg_idle_count` cycles.

`cfg_cg_enable` arms that gating. It is NOT a monitor kill-switch: with it low the clock simply free-runs and the monitor behaves exactly like the base module, packets included. Drive it low for any test that wants the base module's timing without gating effects; drive it high to exercise the gating itself, and expect the counters and any trigger pulse to be lost while the clock is stopped (see the warning under Performance Monitoring).

6.8 Timing Characteristics

6.8.1 Buffer Depths and Latency

Parameter	Default	Channel
`SKID_DEPTH_AR`	2 entries	Skid depth on the AR channel
`SKID_DEPTH_R`	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the

depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

6.9 Related Modules

- **axil4_master_rd_mon** - Base module (functional specification)
- **axil4_master_wr_mon_cg** - Companion monitor wrapper
- **axi_monitor_base** - Core monitoring infrastructure
- **axi_monitor_filtered** - Filtering capabilities
- **AXIL4 Clock-Gated Variants Guide** - The transport-level `_cg` modules, which do perform real clock gating

---

Last Updated: 2026-07-19

6.10 Navigation

- **← Back to Base Module**
- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

7 AXIL4 Master Write

Module: `axil4_master_wr.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

7.1 Overview

The AXIL4 Master Write module gives a master device a buffered AXI4-Lite write interface. AXI4-Lite is the simplified subset of AXI4 designed for control-register access: reduced signal count, single-beat transactions only.

7.1.1 Key Features

- **AXI4-Lite Protocol:** Simplified write-only interface (AW, W, B channels)

- **Single-Beat Transactions:** No burst support (always 1 transfer)
- **Buffered Channels:** Configurable skid buffers for all 3 channels
- **Byte Strobes:** WSTRB support for partial word writes
- **Elastic Buffering:** Decouples frontend and backend timing
- **Activity Monitoring:** Busy signal for clock gating
- **Minimal Latency:** 1-cycle buffer overhead per channel

7.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	Address bus width (typically 32)
`AXIL_DATA_WIDTH`	int	32	Data bus width (32 or 64)
`SKID_DEPTH_AW`	int	2	AW channel skid buffer depth, in entries
`SKID_DEPTH_W`	int	2	W channel skid buffer depth, in entries
`SKID_DEPTH_B`	int	2	B channel skid buffer depth, in entries

7.2.1 Derived Parameters

Each channel's skid buffer stores the whole channel payload as one packed vector, so the derived `*Size` parameters are just the sum of the widths of the signals concatenated into that buffer.

Parameter	Calculation	Description
`AW`	AXIL_ADDR_WIDTH	Internal address width alias
`DW`	AXIL_DATA_WIDTH	Internal data width alias
`AWSize`	AW+3	AW packet: `{awaddr[AW-`

Parameter	Calculation	Description
		1:0], awprot[2:0]} — the 3 is AWPROT
WSize	DW+(DW/8)	W packet: {wdata[DW-1:0], wstrb[DW/8-1:0]} — one strobe bit per byte lane
BSize	2	B packet: bresp[1:0] only

7.3 Ports

7.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	Active-low asynchronous reset

7.3.2 Frontend AXI4-Lite Write Interface (Input Side)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|————|——-|————-| | fub_awaddr | Input | AW | Write address | | fub_awprot | Input | 3 | Protection attributes | | fub_awvalid | Input | 1 | Address valid | | fub_awready | Output | 1 | Address ready (from buffer) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|————|——-|————-| | fub_wdata | Input | DW | Write data | | fub_wstrb | Input | DW/8 | Write strobes (byte lane enables) | | fub_wvalid | Input | 1 | Data valid | | fub_wready | Output | 1 | Data ready (from buffer) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|————|——-|————-| | fub_bresp | Output | 2 | Write response (OKAY, SLVERR, DECERR) | | fub_bvalid | Output | 1 | Response valid (from buffer) | | fub_bready | Input | 1 | Response ready |

7.3.3 Backend AXI4-Lite Master Interface (Output Side)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|————|——-|————-| | m_axil_awaddr | Output | AW | Write address (to slave) | | m_axil_awprot | Output | 3 | Protection attributes | | m_axil_awvalid | Output | 1 | Address valid (from buffer) | | m_axil_awready | Input | 1 | Address ready (from slave) |

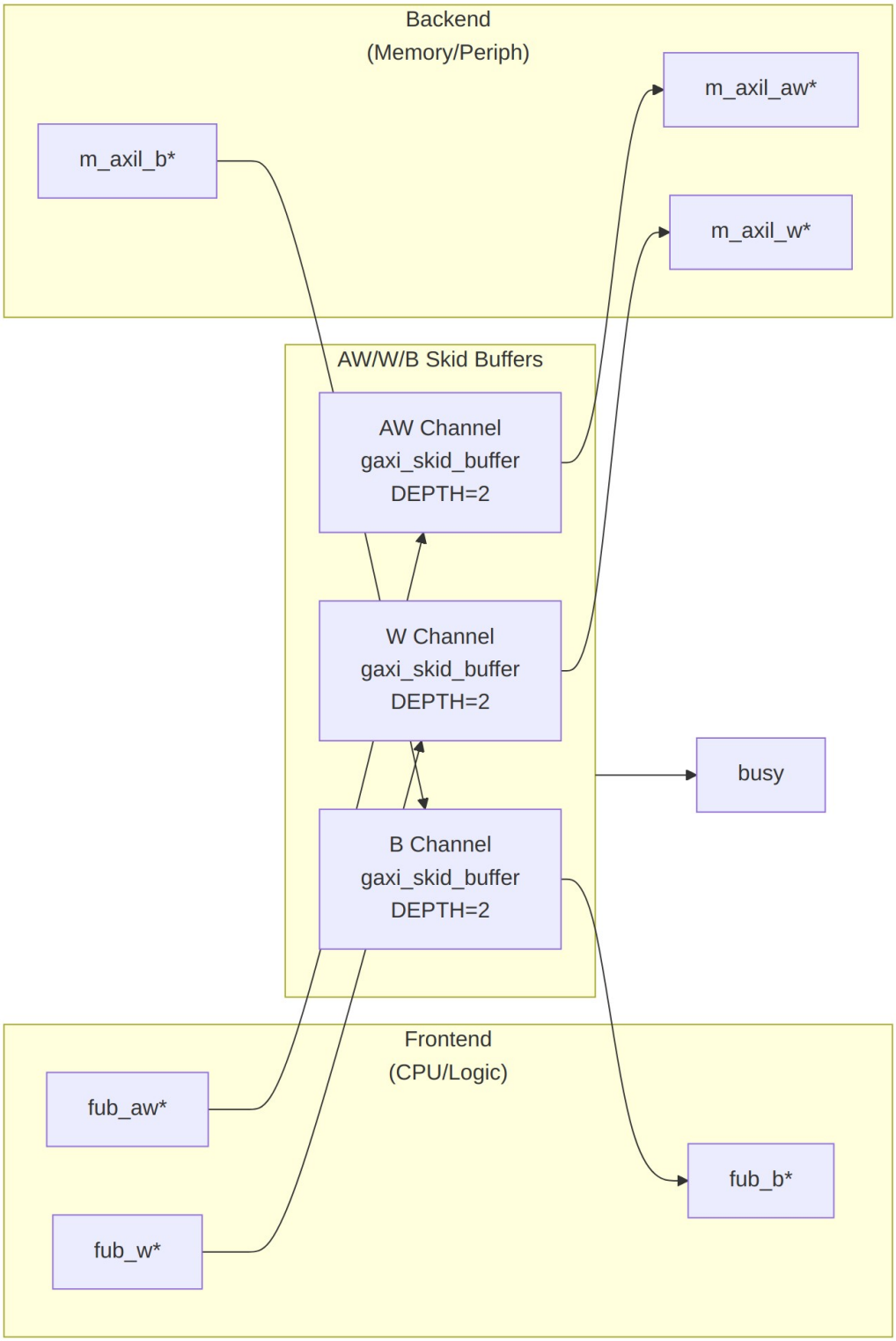
Write Data Channel (W): | Port | Direction | Width | Description | |——|————|——-|————-| | `m_axil_wdata` | Output | DW | Write data (to slave) | | `m_axil_wstrb` | Output | DW/8 | Write strobes | | `m_axil_wvalid` | Output | 1 | Data valid (from buffer) | | `m_axil_wready` | Input | 1 | Data ready (from slave) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|————|——-|————-| | `m_axil_bresp` | Input | 2 | Write response (from slave) | | `m_axil_bvalid` | Input | 1 | Response valid (from slave) | | `m_axil_bready` | Output | 1 | Response ready (to slave) |

7.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Interface active (for clock gating)

7.4 Functional Description



7.4.1 Write Transaction Sequence

AW and W travel in parallel through their own buffers, and the B response comes back on its own — one clock of buffer overhead on each leg:

```
Cycle 1:  fub_awaddr=0x2000, fub_awvalid=1, fub_awprot=0
          fub_wdata=0xCAFEBABE, fub_wstrb=4'b1111, fub_wvalid=1
          fub_awready=1, fub_wready=1 (buffers accept)

Cycle 2:  Buffers forward to backend:
          m_axil_awaddr=0x2000, m_axil_awvalid=1
          m_axil_wdata=0xCAFEBABE, m_axil_wstrb=4'b1111,
m_axil_wvalid=1
          m_axil_awready=1, m_axil_wready=1 (slave accepts)

Cycle 3-5: Slave processes write

Cycle 6:  m_axil_bresp=OKAY, m_axil_bvalid=1
          m_axil_bready=1 (B buffer accepts)

Cycle 7:  B buffer forwards to frontend:
          fub_bresp=OKAY, fub_bvalid=1
          fub_bready=1 (frontend accepts)
```

7.5 Timing Characteristics

7.5.1 Latency

Path	Cycles	Notes
Frontend → Backend (AW)	1	Skid buffer overhead
Frontend → Backend (W)	1	Skid buffer overhead
Backend → Frontend (B)	1	Skid buffer overhead
Total write latency	Slave latency + 2	AW and W traverse in PARALLEL (separate channels, one cycle), then the slave, then B.

Path	Cycles	Notes
		Best case

The +2 best case assumes the slave asserts AWREADY and WREADY in the **same** cycle, so the AW and W buffer traversals overlap: one cycle for that pair, not one each. (This section said +3, which is the figure you get by adding both hops – the arithmetic the sentence right here rules out.) If the slave accepts the address and the data in different cycles, the write completes off the later of the two handshakes, and the total is that skew plus the slave and B latency. Backpressure on B adds further cycles.

7.5.2 Throughput

Scenario	Rate	Notes
Back-to-back writes	1 write/cycle	If buffers not stalled
Typical peripheral	1 write/N cycles	N = slave response time

7.5.3 Resource Usage

These are **order-of-magnitude estimates**, not measured synthesis results — no target device, tool version, or optimization setting is attached to them. They scale with `AXIL_DATA_WIDTH` and the `SKID_DEPTH_*` settings. Synthesize for your own device before budgeting area.

Resource	Count	Notes
LUTs	~300	Estimate, 32-bit data, 3 channels, default depths
FFs	~250	Buffer state + control
BRAM	0	Skid buffers are flop-based

7.5.4 Buffer Depths and Latency

Parameter	Default	Channel
`SKID_DEPTH_AW`	2 entries	Skid depth on the AW channel
`SKID_DEPTH_W`	2 entries	Skid depth on the W channel
`SKID_DEPTH_B`	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

7.6 Usage Examples

7.6.1 Basic Integration

```
axil4_master_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),      // 2 entries
    .SKID_DEPTH_W(2),       // 2 entries
    .SKID_DEPTH_B(2)        // 2 entries
) u_axil_master_wr (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),

    // Frontend interface (from CPU)
    .fub_awaddr     (cpu_awaddr),
    .fub_awprot     (cpu_awprot),
    .fub_awvalid    (cpu_awvalid),
    .fub_awready    (cpu_awready),

    .fub_wdata      (cpu_wdata),
    .fub_wstrb      (cpu_wstrb),
    .fub_wvalid     (cpu_wvalid),
    .fub_wready     (cpu_wready),

    .fub_bresp      (cpu_bresp),
    .fub_bvalid     (cpu_bvalid),
    .fub_bready     (cpu_bready),

    // Backend interface (to peripheral/memory)
    .m_axil_awaddr  (periph_awaddr),
    .m_axil_awprot  (periph_awprot),
    .m_axil_awvalid (periph_awvalid),
    .m_axil_awready (periph_awready),

    .m_axil_wdata   (periph_wdata),
    .m_axil_wstrb   (periph_wstrb),
    .m_axil_wvalid  (periph_wvalid),
```

```
    .m_axil_wready  (periph_wready),

    .m_axil_bresp   (periph_bresp),
    .m_axil_bvalid  (periph_bvalid),
    .m_axil_bready  (periph_bready),

    // Status
    .busy           (wr_busy)
);
```

7.6.2 Register Write Example (Testbench Stimulus)

The two snippets below are **testbench stimulus, not RTL**. They use an `initial` block and blocking assignments to drive the frontend ports and are not synthesizable. In a real design the frontend handshake is driven by an `always_ff` block or by the register-access state machine of the requesting logic.

```
// Write to control register at 0x2000
initial begin
    @(posedge aclk);

    // Present address and data simultaneously
    fub_awaddr  = 32'h2000;
    fub_awprot  = 3'b000;
    fub_awvalid = 1'b1;

    fub_wdata   = 32'hDEADBEEF;
    fub_wstrb   = 4'b1111;  // All bytes valid
    fub_wvalid  = 1'b1;

    @(posedge aclk);
    while (!(fub_awready && fub_wready)) @(posedge aclk);
    fub_awvalid = 1'b0;
    fub_wvalid  = 1'b0;

    // Wait for write response
    fub_bready = 1'b1;
    @(posedge aclk);
    while (!fub_bvalid) @(posedge aclk);

    $display("Write response: BRESP=%0d", fub_bresp);
end
```

7.6.3 Partial Write Example (Byte Strobes)

```
// Write only upper 16 bits to 32-bit register
fub_awaddr  = 32'h2004;
```

```
fub_awprot  = 3'b000;
fub_awvalid = 1'b1;

fub_wdata   = 32'hABCD0000;  // Data in upper half
fub_wstrb   = 4'b1100;       // Only upper 2 bytes valid
fub_wvalid  = 1'b1;

// Result: Register[31:16] = 0xABCD, Register[15:0] unchanged
```

7.7 Design Notes

7.7.1 AXI4-Lite Protocol Constraints

Single-Beat Only: - No burst support (implicitly AWLEN=0, AWSIZE=log2(DW/8)) - Each AW+W transaction results in exactly 1 B response - No WLAST signal (always last and only beat)

No Out-of-Order: - No AWID/BID signals - Responses always in order - AW and W can arrive in any order as far as **this module** is concerned: the AW and W skid buffers are fully independent and nothing here pairs an address with its data. The protocol permits either order, but a downstream slave may still require AW before W (or at least in the same cycle) to decode the target before it can consume the data. Check the slave's requirements — this module will not reorder for you.

Reduced Signals: - No AWCACHE, AWQOS, AWREGION, AWLOCK, AWSIZE, AWLEN, AWBURST - Only AWADDR and AWPROT for control

7.7.2 Buffer Depth Selection

`SKID_DEPTH_AW`, `SKID_DEPTH_W`, and `SKID_DEPTH_B` are passed straight through to the `DEPTH` parameter of `gaxi_skid_buffer`. `DEPTH` is the **literal entry count**, not an exponent, and is constrained to one of 2..8 inclusive.

AW Channel (Addresses): - **SKID_DEPTH_AW** = **2** (2 entries): Typical for control registers - Increase if many back-to-back writes with slow slave

W Channel (Data): - **SKID_DEPTH_W** = **2** (2 entries): Matches AW depth - Should be >= SKID_DEPTH_AW to avoid W channel stalls

B Channel (Responses): - **SKID_DEPTH_B** = **2** (2 entries): Responses are single-beat - Smaller depth acceptable if frontend can always accept

7.7.3 Write Strobe (WSTRB) Usage

32-bit Data Width: | WSTRB[3:0] | Bytes Written | Use Case | |————|————|———-| | 4'b0001 | [7:0] | Byte write to offset 0 | | 4'b0010 | [15:8] | Byte write to offset 1 | | 4'b0011 |

[15:0] | Half-word write | | 4'b1100 | [31:16] | Upper half-word | | 4'b1111 | [31:0] | Full word write |

64-bit Data Width: | WSTRB[7:0] | Bytes Written | Use Case | |————|————|———-| | 8'b0000_0001 | [7:0] | Byte write | | 8'b0000_1111 | [31:0] | Lower word | | 8'b1111_0000 | [63:32] | Upper word | | 8'b1111_1111 | [63:0] | Full double-word |

7.7.4 Busy Signal

`busy` is the OR of three persistent terms and two transient terms:

Term	Kind	Condition
AW buffer occupancy	Persistent	`int_aw_count > 0`
W buffer occupancy	Persistent	`int_w_count > 0`
B buffer occupancy	Persistent	`int_b_count > 0`
Frontend presenting a new transaction	Transient	`fub_awvalid || fub_wvalid`
Backend presenting a response	Transient	`m_axil_bvalid`

The two transient terms can be high for a single cycle, so `busy` itself can pulse. This is intentional: the clock-gate controller does not gate on `busy` directly. `amba_clock_gate_ctrl` registers activity into a `wakeup` flop and then requires `cfg_cg_idle_count` consecutive idle cycles before gating, so a one-cycle handshake reloads the idle counter rather than causing the gate to oscillate.

Use for: - Clock gating control (see `axil4_master_wr_cg`) - Power management - Interface idle detection

---

7.8 Related Modules

7.8.1 Core Modules

- **axil4_master_rd** - Read master counterpart
- **axil4_slave_rd** - Slave read interface
- **axil4_slave_wr** - Slave write interface

7.8.2 Monitor Modules

- **axil4_master_wr_mon** - Master write with monitoring (`rtl/amba/monitor/`)

7.8.3 Clock-Gated Variants

- **axil4_master_wr_cg** - Clock-gated version

7.8.4 Supporting Infrastructure

- **gaxi_skid_buffer** - Elastic buffering (rtl/amba/gaxi/)

7.9 References

7.9.1 Specifications

- ARM IHI 0022E: AMBA AXI and ACE Protocol Specification
- Chapter 4: AXI4-Lite (simplified subset)

7.9.2 Documentation

- **rtl-amba Overview** - Complete AMBA subsystem
- **AXI4 Master Write** - Full AXI4 comparison

7.9.3 Source Code

- RTL: `rtl/amba/axil4/axil4_master_wr.sv`
- Tests: `val/amba/test_axil4_master_wr.py`
- Framework: `bin/TBClasses/components/axil4/`

Last Updated: 2026-07-19

7.10 Testing

`val/amba/test_axil4_master_wr.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_master_wr.py -v
```

7.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

8 AXIL4 Master Write Interface (Clock-Gated)

Module: `axil4_master_wr_cg.sv` **Base Module:** axil4_master_wr **Location:** `rtl/amba/axil4/` **Status:** Production Ready

---

8.1 Overview

This is the **clock-gated variant** of axil4_master_wr.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **AXIL4 Clock-Gated Variants Guide** (AXIL4-specific)

→ **Clock-Gated Variants Guide** (cross-protocol overview)

What the wrapper adds over the base module:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the AXIL4 Clock-Gated Variants Guide for the estimate table and its caveats
- **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
- **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module

---

8.2 Parameters

In addition to all axil4_master_wr parameters:

The `_cg` wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no per-domain `CG_GATE_*` parameters on this module.

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle counter; max idle count = 2^N - 1

8.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

8.3 Ports

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating (0 = clock always running)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating
cg_gating	Output	1	Clock currently gated
cg_idle	Output	1	No activity on the previous cycle

The base module's `busy` output is **not** exposed on the `_cg` wrapper; it is consumed internally as a wakeup term. All other ports are identical to `axil4_master_wr`.

8.4 Usage Examples

```
axil4_master_wr_cg #(
    // Base module parameters (see axil4_master_wr.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2),
```

```
    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime, not parameters)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd5),

    // ... all other ports same as axil4_master_wr, except `busy`,
    // which is replaced by cg_gating / cg_idle
    .cg_gating(clk_gated),
    .cg_idle(if_idle)
);
```

8.5 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The transport datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to `axil4_master_wr` and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (upstream valids plus the base module's `busy`) and `axi_valid` (downstream valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `!cg_gating`, so an upstream master sees no acceptance until the clock is running again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

8.6 Timing Characteristics

8.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel

Parameter	Default	Channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

8.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; `cfg_monitor_enable` and the datapath are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the counter, plus one for the `r_wakeup` flop. Budget the idle count against your traffic's inter-burst gap; too small and the block wakes constantly, too large and it never gates.

8.8 Related Modules

- **axil4_master_wr** - Base module functionality
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

8.9 References

- **axil4_clock_gating_guide.md** - AXIL4 Clock Gating Guide
- **clock_gated_variants.md** - Cross-Protocol Clock Gating Guide

---

Last Updated: 2026-07-19

8.10 Testing

`val/amba/test_axil4_master_wr_cg.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 1 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_master_wr_cg.py -v
```

`val/amba/test_cg_peer_ready.py` additionally asserts that this wrapper gates with the peer's READY parked high – the property the activity term exists to satisfy. See `vault/handbook/design/clock-gating-activity-terms.md`.

8.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

9 AXIL4 Master Write with Monitoring

Module: `axil4_master_wr_mon.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

9.1 Overview

Combines **axil4_master_wr** with **axi_monitor_filtered** for write transaction monitoring.

9.1.1 Key Features

- All features of **axil4_master_wr**
- Integrated write monitoring (AW, W, B channels)

- 2-level filtering (packet-type masks, then per-event-code masks) and error detection
- Simplified for AXI4-Lite (MAX_TRANSACTIONS=8)

9.2 Parameters

Identical to **axil4_master_rd_mon** including: - UNIT_ID, AGENT_ID, MAX_TRANSACTIONS, ENABLE_FILTERING, ADD_PIPELINE_STAGE, USE_MONITOR, N_ADDR_RANGES - ACLK_MHZ (default 100) and CFI_MIN_FREQ_MHZ / CFI_MAX_FREQ_MHZ (default ACLK_MHZ) – the microsecond tick LUT. **Leave ACLK_MHZ at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** - USE_WDATA_ORDER_Q (default 0) and NUM_BANKS (default 1) – **NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q=1**, or axi_monitor_trans_mgr fails elaboration - ADDR_FILTER_ENABLE (default 0) – synthesises the address-range report filter; the cfg_addr_filter_* ports arm it at runtime - ACTIVE_TRANS_THRESHOLD (default MAX_TRANSACTIONS/2): active-transaction count that trips a threshold packet when cfg_threshold_enable=1; replaces the former hardwired value - The synthesis-cone parameters ENABLE_ERROR_LOGIC, ENABLE_TIMEOUT_LOGIC, ENABLE_COMPL_LOGIC, ENABLE_THRESHOLD_LOGIC, ENABLE_PERF_LOGIC (all default 1) and ENABLE_DEBUG_LOGIC (default 0), each of which drops the matching detection cone when set to 0. ENABLE_PERF_PACKETS is tied 1'b1 and ENABLE_DEBUG_MODULE 1'b0 internally. - ID_FILTER_ENABLE (default 0), ID_MATCH_BASE (default 0) and ID_MATCH_COUNT (default 0 = all IDs) – the per-instance ID-slice filter, inherited from the shared monitor core. **Leave ID_FILTER_ENABLE at 0 on AXI4-Lite.** This wrapper hardwires cmd_id/data_id/resp_id to 1'b0 because AXI4-Lite has no ID signals, so enabling the filter with an ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and silently drops ALL monitoring rather than narrowing it - SKID_DEPTH_AW (default 2), SKID_DEPTH_W (default 2), SKID_DEPTH_B (default 2) – skid-buffer depth per channel. Legal range 2..8 inclusive; odd depths are legal

See **axil4_master_rd_mon** for complete parameter descriptions. The removed CAM_PIPELINE / TRANS_CAM_PIPELINE parameters no longer exist (the CAM is always pipelined).

9.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

9.3 Ports

Same as **axil4_master_rd_mon**: - Monitor configuration inputs, including `cfg_error_enable`, `cfg_timeout_enable`, `cfg_compl_enable`, `cfg_threshold_enable`, `cfg_perf_enable`, `cfg_debug_enable` - `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Filtering masks (9: eight `cfg_axi_*_mask` plus the reserved `cfg_axi_err_select`) - The performance-monitoring config/status ports (see Performance Monitoring and the read-monitor port table) - `cfg_freq_sel` (Input, 4) - `counter_freq_invariant` LUT index scaling the 1 us timer tick; the microsecond timeouts are measured in these ticks. With the default `CFI_MIN_FREQ_MHZ == CFI_MAX_FREQ_MHZ == ACLK_MHZ` every LUT entry is identical, so it has no effect until you give CFI a real MIN..MAX range - Monitor bus output (valid/ready/data) - Status outputs (busy, active_transactions, error_count)

9.4 Functional Description

9.4.1 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of W-channel (write-data) utilization counters to `axi_monitor_base`. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals. `cfg_perf_enable` does NOT gate them: it selects the window start/end event (it is edge-detected for `cfg_start_event_sel` modes 010/011) and enables the perf PACKET class. The counters themselves advance whenever a window is open, enabled or not. `ENABLE_PERF_LOGIC = 0` does NOT drop them: the window FSM and its counters are unconditional `always_ff` blocks in `axi_monitor_base`, outside every generate. That parameter gates only `g_perf` in the reporter – the legacy perf-packet cone and the two lifetime counters. `USE_MONITOR = 0` is what ties the perfmon outputs off.

> Avoid enabling completion (`cfg_compl_enable`) and performance (`cfg_perf_enable`) packets simultaneously under heavy traffic — the monitor bus sustains at most one packet per two cycles. Runtime-disabling either class is safe (terminal entries auto-retire; see axi_monitor_reporter); alternatively, `cfg_axi_pkt_mask` drops the packets while keeping marking and counting. See `docs/user-guides/AXI_Monitor_Configuration_Guide.md`.

9.4.1.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit) select the event source (e.g. `3'b010` selects the `cfg_perf_enable` edge).
- `cfg_start_trigger` / `cfg_end_trigger` pulses fire the window directly from an engine or CSR.
- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

9.4.1.2 Utilization Counters (W channel)

Every in-window cycle is classified by the W-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	wvalid && wready	productive beat transferred
`perf_bp_cycles`	32	wvalid && !wready	back-pressure (data offered, consumer not ready)
`perf_starv_cycles`	32	!wvalid && wready	starvation (consumer ready, no valid data)
`perf_idle_cycles`	32	!wvalid && !wready	idle

The four buckets sum to `window_cycles`, so utilization = `perf_prod_cycles / window_cycles`.

9.4.1.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats × (1 << AxSIZE); AXIL is fixed at `AxSIZE=3'b010` (4 bytes)
`perf_burst_count`	32	AW (write-address) handshakes

AXI4-Lite note: every transaction is a single data beat (AWLEN is implicitly 0), so `perf_burst_count` counts AW handshakes = transactions and `perf_beat_count` equals the transaction count. Average burst length (`perf_beat_count / perf_burst_count`) is therefore always 1.

9.4.1.4 Perfmon & Additional Control Ports

The wrapper adds the perfmon config/status ports (`cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`, `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`) and the completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) with the same directions/widths and semantics as **axil4_master_rd_mon** — the only difference is that `perf_burst_count` counts AW handshakes and the utilization buckets watch the W channel. As on the read monitor, `cfg_debug_level`/`cfg_debug_mask`/`cfg_active_trans_threshold` are tied to constants internally and not exposed.

9.4.2 Monitor Backpressure (block_ready)

`block_ready` is a flow-control net inside the wrapper, and it IS brought out: `debug_block_ready` is an output port of this module (the `_cg` wrapper ties it off, so use the base module when you need the tap). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `fub_axil_awready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `fub_axil_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property, `ap_disabled_never_stalls`; this module has no `ifdef FORMAL` block of its own, so here the guarantee rests on the gate expression above, not a proof).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `monitor_common_pkg`), and `block_ready` re-asserts at occupancy `< MAX_TRANSACTIONS - (reserve - 1)` – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified

by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

9.4.3 Address-Range Checker

Identical to **axil4_master_rd_mon** except the checker watches AW (write address) handshakes instead of AR (read address) handshakes. The `cfg_addr_*` configuration inputs and monbus event encoding are the same. See the read monitor's Address-Range Checker section for full details.

9.4.4 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

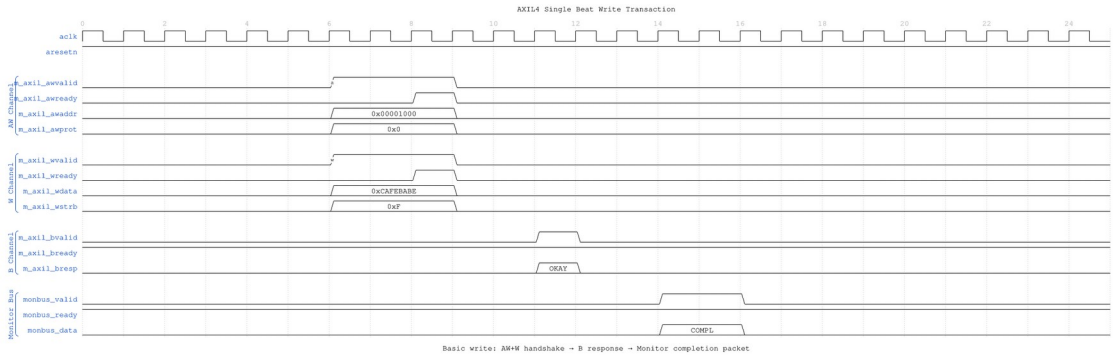
Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is what makes it safe to reprogram live. Contrast the address-range CHECKER above, which re-evaluates per command.

There is no runtime ID filter here. AXI-Lite has no transaction IDs, so this wrapper ties `cfg_id_filter_enable` / `cfg_id_match_base` / `cfg_id_match_count` off on the inner monitor rather than exposing them — a filter keyed on a field the protocol lacks has nothing to match against.

9.5 Waveforms

9.5.1 Scenario 1: Single-Beat Write Transaction

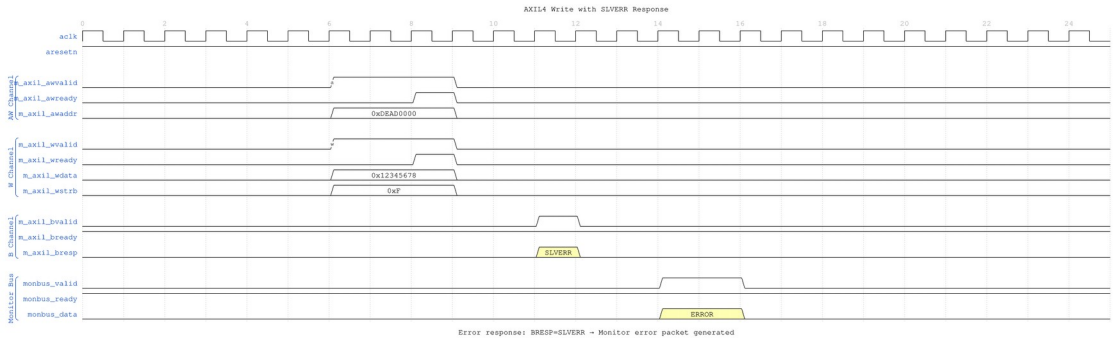


Single Beat Write

WaveJSON: single_beat_write_001.json

Key Observations: - AW and W channels handshake simultaneously (common in AXIL) - B channel response: Slave returns BRESP=OKAY - Monitor generates completion packet when B phase completes - WSTRB indicates which byte lanes are valid

9.5.2 Scenario 2: Write Error (SLVERR)

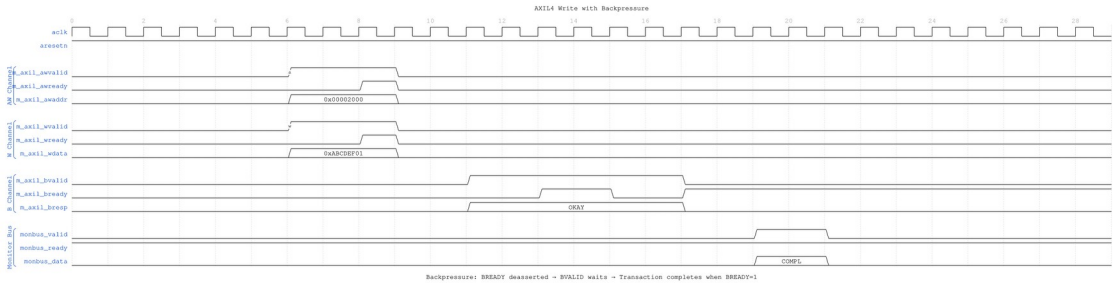


Write Error SLVERR

WaveJSON: write_error_slverr_001.json

Key Observations: - Invalid address triggers BRESP=SLVERR - Monitor detects error response and generates ERROR packet - Write data sent but not committed by slave - Error packet includes address and response code

9.5.3 Scenario 3: Write with Backpressure



Write Backpressure

WaveJSON: write_backpressure_001.json

Key Observations: - Master not ready: BREADY deasserted - Slave holds BVALID until BREADY=1 - Monitor tracks extended write latency - Completion packet generated after B handshake

---

9.6 Usage Examples

```
axil4_master_wr_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(1),
    .AGENT_ID(11),  // Different from read monitor
    .MAX_TRANSACTIONS(8)
) u_axil_wr_mon (
    // AXIL write interfaces (AW, W, B)
    // Monitor configuration and bus
    // Same pattern as axil4_master_rd_mon
);
```

9.7 Timing Characteristics

9.7.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B

Parameter	Default	Channel
		channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

9.8 Design Notes

Monitor cost is not incremental. The transaction table is `bus_transaction_t` x `MAX_TRANSACTIONS`, and the reporter keeps a second full copy (`r_trans_table_local`), so a monitored interface is a multiple of the unmonitored one rather than a few percent on top. `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`perf_byte_count` scales with the bus width. `cmd_size` is derived as `$clog2(AXIL_DATA_WIDTH/8)`. It was hardwired to `3'b010` (4 bytes) until 2026-09-02, which halved every byte count on a 64-bit Lite bus without failing anything. `val/amba/test_axil_perf_byte_count.py` pins it at both legal widths.

Do not enable completion and performance packets together under load. The monitor bus sustains at most one packet per two cycles. Use `cfg_axi_pkt_mask` to drop a class while keeping its marking and counting.

The ID filter is inert and must stay that way. AXI4-Lite has no IDs, so this wrapper ties the monitor's ID inputs to zero; enabling `ID_FILTER_ENABLE` with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring rather than narrowing it.

9.9 Related Modules

- **axil4_master_wr** - Base functional module
- **axil4_master_rd_mon** - Read monitor counterpart
- **AXI4 Master Write Mon** - Full AXI4 reference

Last Updated: 2026-07-19

9.10 Testing

`val/amba/test_axil4_master_wr_mon.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_master_wr_mon.py -v
```

9.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**

10 AXIL4 Master Write Monitor (Clock-Gated)

Module: `axil4_master_wr_mon_cg.sv` **Base Module:** axil4_master_wr_mon **Location:** `rtl/amba/axil4/` **Status:** Partial — see Implementation Status

---

10.1 Overview

`axil4_master_wr_mon_cg` wraps axil4_master_wr_mon and adds a power-management control and status interface. All monitoring, filtering, address-range checking, and performance-monitoring behavior is that of the base module; see axil4_master_wr_mon.md for the complete functional specification.

10.1.1 Implementation Status

This wrapper gates the monitor's clock for real. It instantiates `amba_clock_gate_ctrl`, and the base `axil4_master_wr_mon` inside it is driven from `gated_aclk`, not from `aclk`.

What it does:

1. **Gates the clock.** `amba_clock_gate_ctrl` counts `cfg_cg_idle_count` idle cycles and stops `gated_aclk`, reporting on `cg_gating` (the gated clock is stopped) and `cg_idle` (no activity observed).
2. **Holds off the interfaces while gated.** `fub_axil_awready`, `fub_axil_wready` and `m_axil_bready` are forced low under `cg_gating`, so nothing is accepted into a stopped clock.

3. **Masks the monbus valid while gated** (`monbus_valid = w_monbus_valid && !cg_gating`), which is what makes delivery exactly-once across a gating edge rather than a held valid replayed on wake.

Two earlier versions of this page were wrong in opposite directions, so both are worth naming. One described a `cg_cycles_saved` output, a `cfg_cg_idle_threshold` input and `ENABLE_CLOCK_GATING` / `CG_IDLE_CYCLES` parameters; none of those exist anywhere in `rtl/amba`. The other – the correction to the first – concluded that the wrapper therefore gates nothing and "will not reduce dynamic power". That was the more damaging error: it is false, and it steers an integrator away from the right part. The real controls are `cfg_cg_enable` and `cfg_cg_idle_count` (width `CG_IDLE_COUNT_WIDTH`); there is no cycles-saved counter of any kind.

Gating behaviour is asserted directly by `val/amba/test_mon_cg_gating.py` (phase 5 for the ready hold-off, phase 6 for exactly-once monbus delivery).

10.2 Parameters

In addition to all axil4_master_wr_mon parameters (including `USE_MONITOR` and `N_ADDR_RANGES`):

Parameter	Type	Default	Description
`ACLK_MHZ`	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was unreachable through this wrapper until 2026-09-01.
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	Lowest frequency the tick LUT must cover (dynamic-frequency builds).
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	Highest frequency the

Parameter	Type	Default	Description
			tick LUT must cover.
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue. Required (=1) whenever NUM_BANKS > 1.
NUM_BANKS	int	1	Transaction-table banking. >**1 on this WRITE monitor requires USE_WDATA_ORDER_Q=1** – axi_monitor_trans_mgr fails elaboration otherwise (the WID-less fallback double-counts one W beat across banks).
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves cfg_addr_filter_enable low filters nothing and looks broken.
CG_IDLE_COUNT_WIDTH	int	4	Width of cfg_cg_idle_count; sets the longest programmable idle threshold
ADD_PIPELINE_STAGE	bit	0	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels

Parameter	Type	Default	Description
			(packet type, then event code). Level 2 is reserved and routes nothing.

There are no CG_GATE_MONITOR, CG_GATE_REPORTER, or CG_GATE_TIMERS parameters, and no independent gating domains. Earlier revisions of this document described such a scheme; it was never implemented.

10.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

10.3 Ports

10.3.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
cfg_addr_filter_enable	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever ADDR_FILTER_ENABLE says
cfg_addr_filter_low	Input	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	Input	ADDR_WIDTH	Window limit, inclusive

There is no runtime ID filter on AXI4-Lite: the protocol has no IDs to match.

Base-module ports are forwarded unchanged EXCEPT debug_block_ready, which this wrapper does not bring out (use the base module when you need that tap). That includes the cam_clear

control input (Input, 1) - synchronous clear of the monitor transaction CAM, driven from the harness clear control bit, e.g. CTRL[4]. The wrapper adds four ports:

Port	Direction	Width	Description
`cfg_cg_enable`	Input	1	Arms clock gating. It reaches `amba_clock_gate_ctrl` only – `cfg_monitor_enable` is forwarded to the inner monitor untouched, so 0 here does NOT disable the monitor, it just leaves the clock free-running.
`cfg_cg_idle_count`	Input	`CG_IDLE_COUNT_WIDTH`	Idle cycles before the clock gates. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity (one extra for the `r_wakeup` flop).
`cg_gating`	Output	1	High while the clock is gated.
`cg_idle`	Output	1	High while the activity terms are quiet.

The base module's `busy` output remains available on this wrapper.

10.4 Functional Description

10.4.1 Performance Monitoring

The wrapper forwards the base module's full performance-monitoring interface to `axi_monitor_base` **unchanged** — the power-management interface neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for axil4_master_wr_mon:

- **Config inputs:** `cfg_perf_enable`, `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`

- **Status / counters:** `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`

The completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) and the synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) are likewise forwarded unchanged. The utilization buckets watch the **W** (write-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AW handshakes = transactions.

> Note that `cfg_cg_enable = 0` does NOT disable the monitor. It only disarms clock gating, leaving the inner monitor on a free-running clock and behaving exactly like the base module. Use `cfg_monitor_enable = 0` to actually stop monitoring.

10.5 Usage Examples

```
axil4_master_wr_mon_cg #(
    // Base module parameters (see axil4_master_wr_mon.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2),

    // Monitor parameters
    .UNIT_ID(8'h01),
    .AGENT_ID(16'h000A),
    .MAX_TRANSACTIONS(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Power-management interface
    .cfg_cg_enable(1'b1),           // arm gating; 0 = clock free-
runs
    .cfg_cg_idle_count(4'd4),       // idle cycles before the clock
stops
    .cg_gating(cg_gating),          // high while the gated clock is
stopped
    .cg_idle(cg_idle),
```

```
    // ... all other ports same as axil4_master_wr_mon
);
```

10.6 Design Notes

Monitor cost is not incremental. The transaction table is `bus_transaction_t` x `MAX_TRANSACTIONS`, and the reporter keeps a second full copy (`r_trans_table_local`), so a monitored interface is a multiple of the unmonitored one rather than a few percent on top. `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`perf_byte_count` scales with the bus width. `cmd_size` is derived as `$clog2(AXIL_DATA_WIDTH/8)`. It was hardwired to `3'b010` (4 bytes) until 2026-09-02, which halved every byte count on a 64-bit Lite bus without failing anything. `val/amba/test_axil_perf_byte_count.py` pins it at both legal widths.

Do not enable completion and performance packets together under load. The monitor bus sustains at most one packet per two cycles. Use `cfg_axi_pkt_mask` to drop a class while keeping its marking and counting.

The ID filter is inert and must stay that way. AXI4-Lite has no IDs, so this wrapper ties the monitor's ID inputs to zero; enabling `ID_FILTER_ENABLE` with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring rather than narrowing it.

10.7 Testing

A clock IS gated here – that is the wrapper's entire purpose. `amba_clock_gate_ctrl` drives the inner monitor's clock, and it stops when the activity terms go quiet for `cfg_cg_idle_count` cycles.

`cfg_cg_enable` arms that gating. It is NOT a monitor kill-switch: with it low the clock simply free-runs and the monitor behaves exactly like the base module, packets included. Drive it low for any test that wants the base module's timing without gating effects; drive it high to exercise the gating itself, and expect the counters and any trigger pulse to be lost while the clock is stopped (see the warning under Performance Monitoring).

10.8 Timing Characteristics

10.8.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

10.9 Related Modules

- **axil4_master_wr_mon** - Base module (functional specification)
- **axil4_master_rd_mon_cg** - Companion monitor wrapper
- **axi_monitor_base** - Core monitoring infrastructure
- **axi_monitor_filtered** - Filtering capabilities
- **AXIL4 Clock-Gated Variants Guide** - The transport-level `_cg` modules, which do perform real clock gating

---

Last Updated: 2026-07-19

10.10 Navigation

- **← Back to Base Module**
- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**

- ← Back to Main Documentation Index

11 AXIL4 Slave Read

Module: `axil4_slave_rd.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

11.1 Overview

The AXIL4 Slave Read module gives a slave device (memory, peripheral) a buffered AXI4-Lite read interface. It accepts read requests from an interconnect/master and forwards them to backend logic, with elastic buffering for timing closure on both sides.

11.1.1 Key Features

- **AXI4-Lite Slave:** Buffered read-only interface (AR and R channels)
- **Single-Beat Transactions:** No burst support
- **Elastic Buffering:** Decouples interconnect and backend timing
- **Activity Monitoring:** Busy signal for clock gating
- **Minimal Latency:** 1-cycle buffer overhead per channel

11.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	Address bus width
`AXIL_DATA_WIDTH`	int	32	Data bus width (32 or 64)
`SKID_DEPTH_AR`	int	2	AR channel skid buffer depth, in entries
`SKID_DEPTH_R`	int	4	R channel skid buffer depth, in entries

11.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`ARSize`	`AW+3`
`RSize`	`DW+2`

11.3 Ports

11.3.1 Slave Interface (Input Side from Interconnect)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|————|——-|————-| | `s_axil_araddr` | Input | AW | Read address (from interconnect) | | `s_axil_arprot` | Input | 3 | Protection attributes | | `s_axil_arvalid` | Input | 1 | Address valid | | `s_axil_arready` | Output | 1 | Address ready (buffer status) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|————|——-|————-| | `s_axil_rdata` | Output | DW | Read data (to interconnect) | | `s_axil_rresp` | Output | 2 | Read response | | `s_axil_rvalid` | Output | 1 | Data valid | | `s_axil_rready` | Input | 1 | Data ready |

11.3.2 Backend Interface (Output Side to Memory/Logic)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|————|——-|————-| | `fub_araddr` | Output | AW | Read address (to backend) | | `fub_arprot` | Output | 3 | Protection attributes | | `fub_arvalid` | Output | 1 | Address valid | | `fub_arready` | Input | 1 | Address ready (from backend) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|————|——-|————-| | `fub_rdata` | Input | DW | Read data (from backend) | | `fub_rresp` | Input | 2 | Read response | | `fub_rvalid` | Input | 1 | Data valid | | `fub_rready` | Output | 1 | Data ready |

11.3.3 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Interface active (for clock gating)

11.4 Functional Description

Signal Flow: Interconnect → AR Buffer → Backend → R Buffer → Interconnect

The buffering lets the backend respond at its own pace without stalling the interconnect — the whole point of putting skid buffers on both channels.

11.5 Usage Examples

```
axil4_slave_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4)
) u_axil_slave_rd (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_araddr  (interconnect_araddr),
    .s_axil_arprot  (interconnect_arprot),
    .s_axil_arvalid (interconnect_arvalid),
    .s_axil_arready (interconnect_arready),

    .s_axil_rdata   (interconnect_rdata),
    .s_axil_rresp   (interconnect_rresp),
    .s_axil_rvalid  (interconnect_rvalid),
    .s_axil_rready  (interconnect_rready),

    // Backend interface (to memory/peripheral)
    .fub_araddr     (mem_araddr),
    .fub_arprot     (mem_arprot),
    .fub_arvalid    (mem_arvalid),
    .fub_arready    (mem_arready),

    .fub_rdata      (mem_rdata),
    .fub_rresp      (mem_rresp),
    .fub_rvalid     (mem_rvalid),
    .fub_rready     (mem_rready),

    // Status
    .busy           (rd_busy)
);
```

11.6 Design Notes

- **Use Case:** Memory controllers, peripheral slaves, register blocks

11.7 Timing Characteristics

11.7.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

11.8 Related Modules

- **axil4_slave_wr** - Slave write counterpart
- **axil4_master_rd** - Master read interface
- **axil4_slave_rd_mon** - Slave read with monitoring (`rtl/amba/monitor/`)
- **axil4_slave_rd_cg** - Clock-gated version

Last Updated: 2026-07-19

11.9 Testing

`val/amba/test_axil4_slave_rd.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_slave_rd.py -v
```

11.10 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

12 AXIL4 Slave Read Interface (Clock-Gated)

Module: `axil4_slave_rd_cg.sv` **Base Module:** axil4_slave_rd **Location:** `rtl/amba/axil4/`
Status: Production Ready

12.1 Overview

This is the **clock-gated variant** of axil4_slave_rd.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **AXIL4 Clock-Gated Variants Guide** (AXIL4-specific)

→ **Clock-Gated Variants Guide** (cross-protocol overview)

What the wrapper adds over the base module:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the AXIL4 Clock-Gated Variants Guide for the estimate table and its caveats
- **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
- **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module

---

12.2 Parameters

In addition to all axil4_slave_rd parameters:

The _cg wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no ENABLE_CLOCK_GATING parameter, no CG_IDLE_CYCLES parameter, and no per-domain CG_GATE_* parameters on this module.

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle counter; max idle count = 2^N - 1

12.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

12.3 Ports

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating (0 = clock always running)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating
cg_gating	Output	1	Clock currently gated
cg_idle	Output	1	No activity on the previous cycle

The base module's busy output is **not** exposed on the _cg wrapper; it is consumed internally as a wakeup term. All other ports are identical to axil4_slave_rd.

12.4 Usage Examples

```
axil4_slave_rd_cg #(
    // Base module parameters (see axil4_slave_rd.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime, not parameters)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd5),

    // ... all other ports same as axil4_slave_rd, except `busy`,
    // which is replaced by cg_gating / cg_idle
    .cg_gating(clk_gated),
    .cg_idle(if_idle)
);
```

12.5 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The transport datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to `axil4_slave_rd` and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (upstream valids plus the base module's `busy`) and `axi_valid` (downstream valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `!cg_gating`, so an upstream master sees no acceptance until the clock is running again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

12.6 Timing Characteristics

12.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

12.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; `cfg_monitor_enable` and the datapath are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the counter, plus one for the `r_wakeup` flop. Budget the idle count against your traffic's inter-burst gap; too small and the block wakes constantly, too large and it never gates.

12.8 Related Modules

- **axil4_slave_rd** - Base module functionality
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)

- axil4_master_rd_mon_cg.md (AXIL4 monitor)
- apb4_slave_cg.md (APB interface)

12.9 References

- **axil4_clock_gating_guide.md** - AXIL4 Clock Gating Guide
- **clock_gated_variants.md** - Cross-Protocol Clock Gating Guide

---

Last Updated: 2026-07-19

12.10 Testing

`val/amba/test_axil4_slave_rd_cg.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 1 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_slave_rd_cg.py -v
```

`val/amba/test_cg_peer_ready.py` additionally asserts that this wrapper gates with the peer's READY parked high – the property the activity term exists to satisfy. See `vault/handbook/design/clock-gating-activity-terms.md`.

12.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

13 AXIL4 Slave Read with Monitoring

Module: `axil4_slave_rd_mon.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

13.1 Overview

Combines **axil4_slave_rd** with **axi_monitor_filtered** for slave-side read monitoring.

13.1.1 Key Features

- All features of **axil4_slave_rd**
- Slave-side monitoring (backend latency tracking)
- 2-level filtering (packet-type masks, then per-event-code masks) and error detection
- Simplified for AXI4-Lite (MAX_TRANSACTIONS=8)

13.2 Parameters

Identical to **axil4_master_rd_mon** including `N_ADDR_RANGES`, but typically: - `UNIT_ID = 2` (slaves use different unit ID) - `AGENT_ID = 20` (slave agent IDs) - `USE_MONITOR` (synthesis-time monitor enable) - `ACTIVE_TRANS_THRESHOLD` (default `MAX_TRANSACTIONS/2`): threshold-packet trip point, replaces the former hardwired value - `ENABLE_FILTERING` (default 1) and `ADD_PIPELINE_STAGE` (default 0) - `ACLK_MHZ` (default 100) and `CFI_MIN_FREQ_MHZ` / `CFI_MAX_FREQ_MHZ` (default `ACLK_MHZ`) – the microsecond tick LUT. **Leave `ACLK_MHZ` at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** - `USE_WDATA_ORDER_Q` (default 0) and `NUM_BANKS` (default 1) – **`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`**, or `axi_monitor_trans_mgr` fails elaboration - `ADDR_FILTER_ENABLE` (default 0) – synthesises the address-range report filter; the `cfg_addr_filter_*` ports arm it at runtime - `ID_FILTER_ENABLE` (default 0), `ID_MATCH_BASE` (default 0) and `ID_MATCH_COUNT` (default 0 = all IDs) – the per-instance ID-slice filter, inherited from the shared monitor core. **Leave `ID_FILTER_ENABLE` at 0 on AXI4-Lite.** This wrapper hardwires `cmd_id`/`data_id`/`resp_id` to `1'b0` because AXI4-Lite has no ID signals, so enabling the filter with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and silently drops ALL monitoring rather than narrowing it - `SKID_DEPTH_AR` (default 2), `SKID_DEPTH_R` (default 4) – skid-buffer depth per channel. Legal range 2..8 inclusive; odd depths are legal

Also includes the synthesis-cone parameters `ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC` (all default 1) and `ENABLE_DEBUG_LOGIC` (default 0), each dropping its detection cone when set to 0. `ENABLE_PERF_PACKETS` is tied `1'b1` and `ENABLE_DEBUG_MODULE` `1'b0` internally; the removed `CAM_PIPELINE` / `TRANS_CAM_PIPELINE` parameters no longer exist.

For complete parameter descriptions including `N_ADDR_RANGES` and the synthesis-cone parameters, see **axil4_master_rd_mon**.

13.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

13.3 Ports

Same as **axil4_master_rd_mon**, including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]), the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables, and the performance-monitoring config/status ports (see Performance Monitoring).

`cfg_freq_sel` (Input, 4) is also forwarded: the `counter_freq_invariant` LUT index that scales the 1 us timer tick, in which the microsecond timeouts are measured. With the default `CFI_MIN_FREQ_MHZ == CFI_MAX_FREQ_MHZ == ACLK_MHZ` every LUT entry is identical, so it has no effect until you give CFI a real MIN..MAX range.

13.4 Functional Description

13.4.1 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of R-channel (read-data) utilization counters to `axi_monitor_base`. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals. `cfg_perf_enable` does NOT gate them: it selects the window start/end event (it is edge-detected for `cfg_start_event_sel` modes 010/011) and enables the perf PACKET class. The counters themselves advance whenever a window is open, enabled or not. `ENABLE_PERF_LOGIC = 0` does NOT drop them: the window FSM and its counters are unconditional `always_ff` blocks in `axi_monitor_base`, outside every generate. That parameter gates only `g_perf` in the reporter – the legacy perf-packet cone and the two lifetime counters. `USE_MONITOR = 0` is what ties the perfmon outputs off.

> Avoid enabling completion (`cfg_compl_enable`) and performance (`cfg_perf_enable`) packets simultaneously under heavy traffic — the monitor bus sustains at most one packet per two cycles. Runtime-disabling either class is safe (terminal entries auto-retire; see axi_monitor_reporter); alternatively, `cfg_axi_pkt_mask` drops the packets while keeping marking and counting. See `docs/user-guides/AXI_Monitor_Configuration_Guide.md`.

13.4.1.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit) select the event source (e.g. `3'b010` selects the `cfg_perf_enable` edge).
- `cfg_start_trigger` / `cfg_end_trigger` pulses fire the window directly from an engine or CSR.
- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

13.4.1.2 Utilization Counters (R channel)

Every in-window cycle is classified by the R-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	rvalid && rready	productive beat transferred
`perf_bp_cycles`	32	rvalid && !rready	back-pressure (data offered, consumer not ready)
`perf_starv_cycles`	32	!rvalid && rready	starvation (consumer ready, no valid data)
`perf_idle_cycles`	32	!rvalid && !rready	idle

The four buckets sum to `window_cycles`, so utilization = `perf_prod_cycles / window_cycles`.

13.4.1.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats × (1 << AxSIZE); AXIL is fixed

Output	Width	Meaning
		at AxSIZE=3'b010 (4 bytes)
perf_burst_count	32	AR (read-address) handshakes

AXI4-Lite note: every transaction is a single data beat (ARLEN is implicitly 0), so perf_burst_count counts AR handshakes = transactions and perf_beat_count equals the transaction count. Average burst length (perf_beat_count / perf_burst_count) is therefore always 1.

The perfmon config/status ports and the cfg_compl_enable / cfg_threshold_enable / cfg_debug_enable control inputs have the same directions/widths and semantics as the **read-monitor port table**. As there, cfg_debug_level/cfg_debug_mask/cfg_active_trans_threshold are tied to constants internally and not exposed.

13.4.2 Monitor Backpressure (block_ready)

block_ready is a flow-control net inside the wrapper, and it IS brought out: debug_block_ready is an output port of this module (the _cg wrapper ties it off, so use the base module when you need the tap). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of MAX_TRANSACTIONS; the reporter FIFO depth INTR_FIFO_DEPTH has no path to it). The wrapper ANDs it into the upstream-facing s_axil_aready so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream s_axil_aready is forced low until the monitor drains.
- **When USE_MONITOR=0**: block_ready is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When cfg_monitor_enable=0**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property, ap_disabled_never_stalls; this module has no ifdef FORMAL block of its own, so here the guarantee rests on the gate expression above, not a proof).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS) (reserve = 4 for tables of 16 or more, 0 below; the function lives in monitor_common_pkg), and block_ready re-asserts at occupancy < MAX_TRANSACTIONS - (reserve - 1) – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified

by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

13.4.3 Address-Range Checker

Identical to **axil4_master_rd_mon** — monitors AR (read address) handshakes and emits address-range violation packets. See the master read monitor's section for full details.

13.4.4 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

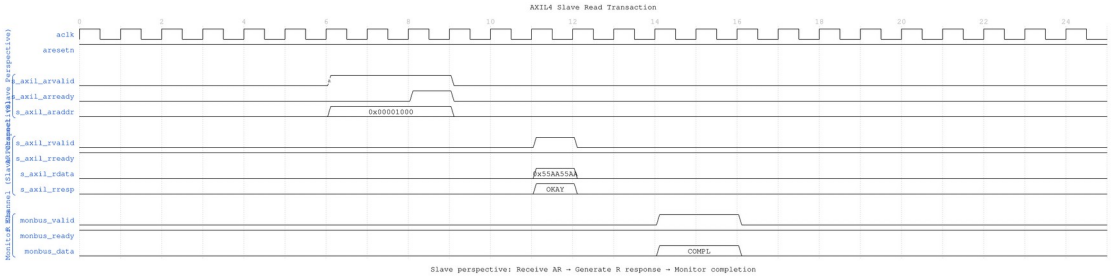
Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is what makes it safe to reprogram live. Contrast the address-range CHECKER above, which re-evaluates per command.

There is no runtime ID filter here. AXI-Lite has no transaction IDs, so this wrapper ties `cfg_id_filter_enable` / `cfg_id_match_base` / `cfg_id_match_count` off on the inner monitor rather than exposing them — a filter keyed on a field the protocol lacks has nothing to match against.

13.5 Waveforms

13.5.1 Scenario 1: Slave Read Transaction

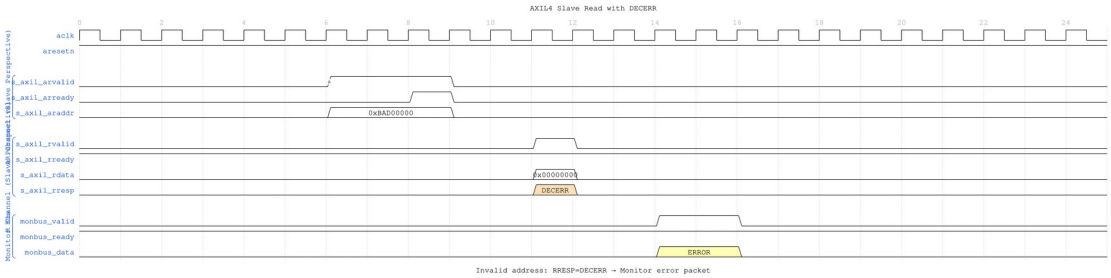


Slave Read Basic

WaveJSON: slave_read_basic_001.json

Key Observations: - Slave perspective: Receive AR request from master - Slave generates R response with data - Monitor tracks backend read latency (AR → R delay) - Completion packet indicates successful read

13.5.2 Scenario 2: Slave Read Error (DECERR)

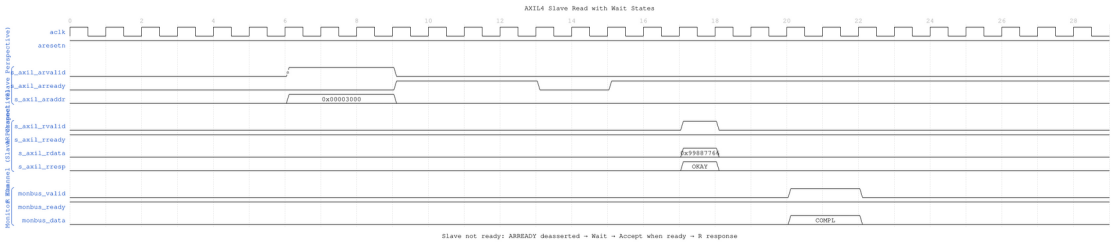


Slave Read DECERR

WaveJSON: slave_read_decerr_001.json

Key Observations: - Invalid address detected by slave address decoder - Slave returns RRESP=DECERR (decode error) - Monitor generates ERROR packet - Data value is don't-care when error occurs

13.5.3 Scenario 3: Slave Read with Wait States



Slave Read Wait

WaveJSON: slave_read_wait_001.json

Key Observations: - Slave not ready: ARREADY deasserted (wait states) - Master holds ARVALID until slave accepts - Backend read takes multiple cycles - Monitor tracks full transaction latency

---

13.6 Usage Examples

```
axil4_slave_rd_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(2),    // Slave unit ID
    .AGENT_ID(20),  // Slave agent ID
    .MAX_TRANSACTIONS(8)
) u_axil_slave_rd_mon (
    // Slave AXIL interfaces (s_axil_*, fub_*)
    // Monitor configuration and bus
);
```

13.7 Timing Characteristics

13.7.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the**

unstalled case – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

13.8 Design Notes

Monitor cost is not incremental. The transaction table is `bus_transaction_t` x `MAX_TRANSACTIONS`, and the reporter keeps a second full copy (`r_trans_table_local`), so a monitored interface is a multiple of the unmonitored one rather than a few percent on top. `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`perf_byte_count` scales with the bus width. `cmd_size` is derived as `$clog2(AXIL_DATA_WIDTH/8)`. It was hardwired to `3'b010` (4 bytes) until 2026-09-02, which halved every byte count on a 64-bit Lite bus without failing anything. `val/amba/test_axil_perf_byte_count.py` pins it at both legal widths.

Do not enable completion and performance packets together under load. The monitor bus sustains at most one packet per two cycles. Use `cfg_axi_pkt_mask` to drop a class while keeping its marking and counting.

The ID filter is inert and must stay that way. AXI4-Lite has no IDs, so this wrapper ties the monitor's ID inputs to zero; enabling `ID_FILTER_ENABLE` with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring rather than narrowing it.

13.9 Related Modules

- **axil4_slave_rd** - Base functional module
- **axil4_slave_wr_mon** - Write monitor counterpart
- **AXI4 Slave Read Mon** - Full AXI4 reference

Last Updated: 2026-07-19

13.10 Testing

`val/amba/test_axil4_slave_rd_mon.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_slave_rd_mon.py -v
```

13.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**

14 AXIL4 Slave Read Monitor (Clock-Gated)

Module: `axil4_slave_rd_mon_cg.sv` **Base Module:** axil4_slave_rd_mon **Location:** `rtl/amba/axil4/` **Status:** Partial — see Implementation Status

---

14.1 Overview

`axil4_slave_rd_mon_cg` wraps axil4_slave_rd_mon and adds a power-management control and status interface. All monitoring, filtering, address-range checking, and performance-monitoring behavior is that of the base module; see axil4_slave_rd_mon.md for the complete functional specification.

14.1.1 Implementation Status

This wrapper gates the monitor's clock for real. It instantiates `amba_clock_gate_ctrl`, and the base `axil4_slave_rd_mon` inside it is driven from `gated_aclk`, not from `aclk`.

What it does:

1. **Gates the clock.** `amba_clock_gate_ctrl` counts `cfg_cg_idle_count` idle cycles and stops `gated_aclk`, reporting on `cg_gating` (the gated clock is stopped) and `cg_idle` (no activity observed).
2. **Holds off the interfaces while gated.** `s_axil_arready` and `fub_axil_rready` are forced low under `cg_gating`, so nothing is accepted into a stopped clock.
3. **Masks the monbus valid while gated** (`monbus_valid = w_monbus_valid && !cg_gating`), which is what makes delivery exactly-once across a gating edge rather than a held valid replayed on wake.

Two earlier versions of this page were wrong in opposite directions, so both are worth naming. One described a `cg_cycles_saved` output, a `cfg_cg_idle_threshold` input and `ENABLE_CLOCK_GATING` / `CG_IDLE_CYCLES` parameters; none of those exist anywhere in `rtl/amba`. The other – the correction to the first – concluded that the wrapper therefore gates nothing and "will not reduce dynamic power". That was the more damaging error: it is false, and it steers an integrator away from the right part. The real controls are `cfg_cg_enable` and `cfg_cg_idle_count` (width `CG_IDLE_COUNT_WIDTH`); there is no cycles-saved counter of any kind.

Gating behaviour is asserted directly by `val/amba/test_mon_cg_gating.py` (phase 5 for the ready hold-off, phase 6 for exactly-once monbus delivery).

14.2 Parameters

In addition to all axil4_slave_rd_mon parameters (including `USE_MONITOR` and `N_ADDR_RANGES`):

Parameter	Type	Default	Description
`ACLK_MHZ`	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was unreachable through this wrapper until 2026-09-01.
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	Lowest frequency the tick LUT must cover (dynamic-frequency builds).
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	Highest frequency the tick LUT must cover.
`USE_WDATA_ORDER_Q`	bit	0	Write-data ordering queue. Required (=1) whenever `NUM_BANKS` > 1.

Parameter	Type	Default	Description
NUM_BANKS	int	1	Transaction-table banking. The USE_WDATA_ORDER_Q pairing rule applies to WRITE monitors only – axi_monitor_trans_mgr guards on (NUM_BANKS > 1) && !IS_READ && ! USE_WDATA_ORDER_Q, so a read monitor may bank freely.
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves cfg_addr_filter_enable low filters nothing and looks broken.
CG_IDLE_COUNT_WIDTH	int	4	Width of cfg_cg_idle_count; sets the longest programmable idle threshold
ADD_PIPELINE_STAGE	bit	0	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing.

There are no CG_GATE_MONITOR, CG_GATE_REPORTER, or CG_GATE_TIMERS parameters, and no independent gating domains. Earlier revisions of this document described such a scheme; it was never implemented.

14.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

14.3 Ports

14.3.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
cfg_addr_filter_enable	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever ADDR_FILTER_ENABLE says
cfg_addr_filter_low	Input	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	Input	ADDR_WIDTH	Window limit, inclusive

There is no runtime ID filter on AXI4-Lite: the protocol has no IDs to match.

Base-module ports are forwarded unchanged EXCEPT debug_block_ready, which this wrapper does not bring out (use the base module when you need that tap). That includes the cam_clear control input (Input, 1) - synchronous clear of the monitor transaction CAM, driven from the harness clear control bit, e.g. CTRL[4]. The wrapper adds four ports:

Port	Direction	Width	Description
cfg_cg_en	Input	1	Arms clock gating. It

Port	Direction	Width	Description
able			reaches amba_clock_gate_ctrl only – cfg_monitor_enable is forwarded to the inner monitor untouched, so 0 here does NOT disable the monitor, it just leaves the clock free-running.
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before the clock gates. The clock stops cfg_cg_idle_count + 2 cycles after the last bus activity (one extra for the r_wakeup flop).
cg_gating	Output	1	High while the clock is gated.
cg_idle	Output	1	High while the activity terms are quiet.

The base module's busy output remains available on this wrapper.

14.4 Functional Description

14.4.1 Performance Monitoring

The wrapper forwards the base module's full performance-monitoring interface to axi_monitor_base **unchanged** — the power-management interface neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for axil4_slave_rd_mon:

- **Config inputs:** cfg_perf_enable, cfg_start_event_sel, cfg_end_event_sel, cfg_start_trigger, cfg_end_trigger, cfg_window_force_close
- **Status / counters:** window_active, window_cycles, perf_prod_cycles, perf_bp_cycles, perf_starv_cycles, perf_idle_cycles, perf_beat_count, perf_byte_count, perf_burst_count

The completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) and the synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) are likewise forwarded unchanged. The utilization buckets watch the **R** (read-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AR handshakes = transactions.

> Note that `cfg_cg_enable = 0` does NOT disable the monitor. It only disarms clock gating, leaving the inner monitor on a free-running clock and behaving exactly like the base module. Use `cfg_monitor_enable = 0` to actually stop monitoring.

14.5 Usage Examples

```
axil4_slave_rd_mon_cg #(
    // Base module parameters (see axil4_slave_rd_mon.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Monitor parameters
    .UNIT_ID(8'h01),
    .AGENT_ID(16'h000A),
    .MAX_TRANSACTIONS(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Power-management interface
    .cfg_cg_enable(1'b1),           // arm gating; 0 = clock free-
runs
    .cfg_cg_idle_count(4'd4),       // idle cycles before the clock
stops
    .cg_gating(cg_gating),          // high while the gated clock is
stopped
    .cg_idle(cg_idle),

    // ... all other ports same as axil4_slave_rd_mon
);
```

14.6 Timing Characteristics

14.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

14.7 Design Notes

Monitor cost is not incremental. The transaction table is `bus_transaction_t` x `MAX_TRANSACTIONS`, and the reporter keeps a second full copy (`r_trans_table_local`), so a monitored interface is a multiple of the unmonitored one rather than a few percent on top. `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`perf_byte_count` scales with the bus width. `cmd_size` is derived as `$clog2(AXIL_DATA_WIDTH/8)`. It was hardwired to `3'b010` (4 bytes) until 2026-09-02, which halved every byte count on a 64-bit Lite bus without failing anything. `val/amba/test_axil_perf_byte_count.py` pins it at both legal widths.

Do not enable completion and performance packets together under load. The monitor bus sustains at most one packet per two cycles. Use `cfg_axi_pkt_mask` to drop a class while keeping its marking and counting.

The ID filter is inert and must stay that way. AXI4-Lite has no IDs, so this wrapper ties the monitor's ID inputs to zero; enabling `ID_FILTER_ENABLE` with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring rather than narrowing it.

14.8 Related Modules

- **axil4_slave_rd_mon** - Base module (functional specification)

- **axil4_slave_wr_mon_cg** - Companion monitor wrapper
- **axi_monitor_base** - Core monitoring infrastructure
- **axi_monitor_filtered** - Filtering capabilities
- **AXIL4 Clock-Gated Variants Guide** - The transport-level `_cg` modules, which do perform real clock gating

---

14.9 Testing

A clock IS gated here – that is the wrapper's entire purpose. `amba_clock_gate_ctrl` drives the inner monitor's clock, and it stops when the activity terms go quiet for `cfg_cg_idle_count` cycles.

`cfg_cg_enable` arms that gating. It is NOT a monitor kill-switch: with it low the clock simply free-runs and the monitor behaves exactly like the base module, packets included. Drive it low for any test that wants the base module's timing without gating effects; drive it high to exercise the gating itself, and expect the counters and any trigger pulse to be lost while the clock is stopped (see the warning under Performance Monitoring).

Last Updated: 2026-07-19

14.10 Navigation

- **← Back to Base Module**
- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

15 AXIL4 Slave Write

Module: `axil4_slave_wr.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

15.1 Overview

The AXIL4 Slave Write module provides a buffered AXI4-Lite write interface for slave devices (memory, peripherals). It accepts write requests from an interconnect/master and forwards them to backend logic with elastic buffering for timing closure.

Key features:

- **AXI4-Lite Slave:** Buffered write-only interface (AW, W, B channels)
- **Single-Beat Transactions:** No burst support
- **Byte Strobes:** WSTRB support for partial writes
- **Elastic Buffering:** Decouples interconnect and backend timing
- **Activity Monitoring:** Busy signal for clock gating
- **Minimal Latency:** 1-cycle buffer overhead per channel

15.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	Address bus width
`AXIL_DATA_WIDTH`	int	32	Data bus width (32 or 64)
`SKID_DEPTH_AW`	int	2	AW channel skid buffer depth, in entries
`SKID_DEPTH_W`	int	2	W channel skid buffer depth, in entries
`SKID_DEPTH_B`	int	2	B channel skid buffer depth, in entries
`BSize`	int	`2 / / resp only`	See the RTL declaration.

15.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AWSize`	`AW+3`
`WSize`	`DW+(DW/8)`

15.3 Ports

15.3.1 Slave Interface (Input Side from Interconnect)

Write Address Channel (AW):

Port	Direction	Width	Description
s_axil_awaddr	Input	AW	Write address (from interconnect)
s_axil_awprot	Input	3	Protection attributes
s_axil_awvalid	Input	1	Address valid
s_axil_awready	Output	1	Address ready (buffer status)

Write Data Channel (W):

Port	Direction	Width	Description
s_axil_wdata	Input	DW	Write data (from interconnect)
s_axil_wstrb	Input	DW/8	Write strobes
s_axil_wvalid	Input	1	Data valid
s_axil_wready	Output	1	Data ready (buffer status)

Write Response Channel (B):

Port	Direction	Width	Description
s_axil_bresp	Output	2	Write response (to interconnect)
s_axil_bvalid	Output	1	Response valid
s_axil_bready	Input	1	Response ready

15.3.2 Backend Interface (Output Side to Memory/Logic)

Write Address Channel (AW):

Port	Direction	Width	Description
fub_awaddr	Output	AW	Write address (to backend)
fub_awprot	Output	3	Protection attributes
fub_awvalid	Output	1	Address valid
fub_awready	Input	1	Address ready (from backend)

Write Data Channel (W):

Port	Direction	Width	Description
fub_wdata	Output	DW	Write data (to backend)
fub_wstrb	Output	DW/8	Write strobes
fub_wvalid	Output	1	Data valid
fub_wready	Input	1	Data ready (from backend)

Write Response Channel (B):

Port	Direction	Width	Description
fub_bresp	Input	2	Write response (from backend)
fub_bvalid	Input	1	Response valid
fub_bready	Output	1	Response ready

15.3.3 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface

Port	Direction	Width	Description
			active (for clock gating)

15.4 Functional Description

Signal flow runs Interconnect → AW/W Buffers → Backend → B Buffer → Interconnect. Each channel crosses its own buffer, so the backend can respond at its own pace without stalling the interconnect — that decoupling is the elastic buffering the Overview mentions, and it's what buys you the timing closure.

15.5 Usage Examples

```
axil4_slave_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_axil_slave_wr (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_awaddr  (interconnect_awaddr),
    .s_axil_awprot  (interconnect_awprot),
    .s_axil_awvalid (interconnect_awvalid),
    .s_axil_awready (interconnect_awready),

    .s_axil_wdata   (interconnect_wdata),
    .s_axil_wstrb   (interconnect_wstrb),
    .s_axil_wvalid  (interconnect_wvalid),
    .s_axil_wready  (interconnect_wready),

    .s_axil_bresp   (interconnect_bresp),
    .s_axil_bvalid  (interconnect_bvalid),
    .s_axil_bready  (interconnect_bready),

    // Backend interface (to memory/peripheral)
    .fub_awaddr     (mem_awaddr),
    .fub_awprot     (mem_awprot),
    .fub_awvalid    (mem_awvalid),
```

```
    .fub_awready    (mem_awready),

    .fub_wdata      (mem_wdata),
    .fub_wstrb      (mem_wstrb),
    .fub_wvalid     (mem_wvalid),
    .fub_wready     (mem_wready),

    .fub_bresp      (mem_bresp),
    .fub_bvalid     (mem_bvalid),
    .fub_bready     (mem_bready),

    // Status
    .busy           (wr_busy)
);
```

15.6 Design Notes

- **Use Case:** Memory controllers, peripheral slaves, register blocks
- **Write Strobes:** Backend must honor WSTRB for partial word writes

15.7 Timing Characteristics

15.7.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

15.8 Related Modules

- **axil4_slave_rd** - Slave read counterpart
- **axil4_master_wr** - Master write interface
- **axil4_slave_wr_mon** - Slave write with monitoring (`rtl/amba/monitor/`)
- **axil4_slave_wr_cg** - Clock-gated version

Last Updated: 2026-07-19

15.9 Testing

`val/amba/test_axil4_slave_wr.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_slave_wr.py -v
```

15.10 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

16 AXIL4 Slave Write Interface (Clock-Gated)

Module: `axil4_slave_wr_cg.sv` **Base Module:** axil4_slave_wr **Location:** `rtl/amba/axil4/`
Status: Production Ready

16.1 Overview

This is the **clock-gated variant** of axil4_slave_wr.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **AXIL4 Clock-Gated Variants Guide** (AXIL4-specific)

→ **Clock-Gated Variants Guide** (cross-protocol overview)

The `axil4_slave_wr_cg` module adds power optimization to `axil4_slave_wr` through activity-based clock gating:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the AXIL4 Clock-Gated Variants Guide for the estimate table and its caveats
- **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
- **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module

---

16.2 Parameters

In addition to all axil4_slave_wr parameters, the `_cg` wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no per-domain `CG_GATE_*` parameters on this module.

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle counter; max idle count = 2^N - 1

16.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AW`	`AXIL_ADDR_WIDTH`
`DW`	`AXIL_DATA_WIDTH`

16.3 Ports

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating (0 = clock always running)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating
cg_gating	Output	1	Clock currently gated
cg_idle	Output	1	No activity on the previous cycle

The base module's busy output is **not** exposed on the _cg wrapper; it is consumed internally as a wakeup term. All other ports are identical to axil4_slave_wr.

16.4 Usage Examples

```
axil4_slave_wr_cg #(
    // Base module parameters (see axil4_slave_wr.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime, not parameters)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd5),

    // ... all other ports same as axil4_slave_wr, except `busy`,
    // which is replaced by cg_gating / cg_idle
    .cg_gating(clk_gated),
    .cg_idle(if_idle)
);
```

16.5 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The transport datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to `axil4_slave_wr` and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (upstream valids plus the base module's `busy`) and `axi_valid` (downstream valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `! cg_gating`, so an upstream master sees no acceptance until the clock is running again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

16.6 Timing Characteristics

16.6.1 Buffer Depths and Latency

Parameter	Default	Channel
`SKID_DEPTH_AW`	2 entries	Skid depth on the AW channel
`SKID_DEPTH_W`	2 entries	Skid depth on the W channel
`SKID_DEPTH_B`	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

16.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; `cfg_monitor_enable` and the datapath are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the counter, plus one for the `r_wakeup` flop. Budget the idle count against your traffic's inter-burst gap; too small and the block wakes constantly, too large and it never gates.

16.8 Related Modules

- **Base Module Functionality:** axil4_slave_wr.md
- **AXIL4 Clock Gating Guide:** axil4_clock_gating_guide.md
- **Cross-Protocol Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

Last Updated: 2026-07-19

16.9 Testing

`val/amba/test_axil4_slave_wr_cg.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 1 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_slave_wr_cg.py -v
```

`val/amba/test_cg_peer_ready.py` additionally asserts that this wrapper gates with the peer's READY parked high – the property the activity term exists to satisfy. See `vault/handbook/design/clock-gating-activity-terms.md`.

16.10 Navigation

- ← Back to AXIL4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

17 AXIL4 Slave Write with Monitoring

Module: `axil4_slave_wr_mon.sv` **Location:** `rtl/amba/axil4/` **Status:** Production Ready

17.1 Overview

Combines **axil4_slave_wr** with **axi_monitor_filtered** for slave-side write monitoring.

Key features:

- All features of **axil4_slave_wr**
- Slave-side write monitoring (AW, W, B channels)
- Backend write latency tracking
- 2-level filtering (packet-type masks, then per-event-code masks) and error detection

17.2 Parameters

In addition to all axil4_slave_wr parameters:

Parameter	Type	Default	Description
`UNIT_ID`	int	2	Unit ID (2 = slaves)
`AGENT_ID`	int	21	Agent ID (slave write agent)
`USE_MONITOR`	bit	—	Synthesis-time monitor enable
`ACTIVE_TRANS_THRESHOLD`	int	`MAX_TRANSACTIONS/2`	Threshold-packet trip point; replaces the former hardwired value

Parameter	Type	Default	Description
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet-type masks, then per-event-code masks)
ADD_PIPELINE_STAGE	bit	0	Insert a register stage for timing closure. Costs a cycle of latency.
ACLK_MHZ	int	100	Clock frequency in MHz; builds the microsecond tick LUT in `counter_freq_invariant`. **Leave `ACLK_MHZ` at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently**
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Lowest frequency the tick LUT must cover (dynamic-frequency builds)
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Highest frequency the tick LUT must cover
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue
NUM_BANKS	int	1	Transaction-table banking. **`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`**, or `axi_monitor_trans_mgr` fails elaboration
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter; the `cfg_addr_filter_*` ports arm it at runtime

Parameter	Type	Default	Description
N_ADDR_RANGES	int	—	Address-range comparator count
ID_FILTER_ENABLE	bit	0	Per-instance ID-slice filter, inherited from the shared monitor core. **Leave ID_FILTER_ENABLE at 0 on AXI4-Lite.** This wrapper hardwires cmd_id/data_id/resp_id to 1'b0 because AXI4-Lite has no ID signals, so enabling the filter with an ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and silently drops ALL monitoring rather than narrowing it
ID_MATCH_BASE	int	0	ID-slice filter base
ID_MATCH_COUNT	int	0	ID-slice filter count (0 = all IDs)
SKID_DEPTH_AW	int	2	AW channel skid-buffer depth, in entries. Legal range 2..8 inclusive; odd depths are legal
SKID_DEPTH_W	int	2	W channel skid-buffer depth, in entries. Legal range 2..8 inclusive; odd depths are legal
SKID_DEPTH_B	int	2	B channel skid-buffer depth, in entries. Legal range 2..8 inclusive; odd depths are legal

Others same as **axil4_master_rd_mon**.

Also includes the synthesis-cone parameters ENABLE_ERROR_LOGIC, ENABLE_TIMEOUT_LOGIC, ENABLE_COMPL_LOGIC, ENABLE_THRESHOLD_LOGIC, ENABLE_PERF_LOGIC (all default 1) and ENABLE_DEBUG_LOGIC (default 0), each dropping its detection cone when set to 0. ENABLE_PERF_PACKETS is tied 1'b1 and ENABLE_DEBUG_MODULE 1'b0 internally; the removed CAM_PIPELINE / TRANS_CAM_PIPELINE parameters no longer exist.

For complete parameter descriptions, see **axil4_master_rd_mon**.

17.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

17.3 Ports

Same as **axil4_master_rd_mon**, including the cam_clear control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]), the cfg_compl_enable / cfg_threshold_enable / cfg_debug_enable control enables, and the performance-monitoring config/status ports (see Performance Monitoring).

cfg_freq_sel (Input, 4) is also forwarded: the counter_freq_invariant LUT index that scales the 1 us timer tick, in which the microsecond timeouts are measured. With the default CFI_MIN_FREQ_MHZ == CFI_MAX_FREQ_MHZ == ACLK_MHZ every LUT entry is identical, so it has no effect until you give CFI a real MIN..MAX range.

17.4 Functional Description

17.4.1 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of W-channel (write-data) utilization counters to axi_monitor_base. All counters accumulate **only while a window is open** (window_active = 1) and hold their values between windows so the host can read a completed window's totals. cfg_perf_enable does NOT gate them: it selects the window start/end event (it is edge-detected for cfg_start_event_sel modes 010/011) and enables the perf PACKET class. The counters themselves advance whenever a window is open, enabled or not. ENABLE_PERF_LOGIC = 0 does NOT drop them: the window FSM

and its counters are unconditional `always_ff` blocks in `axi_monitor_base`, outside every generate. That parameter gates only `g_perf` in the reporter – the legacy perf-packet cone and the two lifetime counters. `USE_MONITOR = 0` is what ties the perfmon outputs off.

> Avoid enabling completion (`cfg_compl_enable`) and performance (`cfg_perf_enable`) packets simultaneously under heavy traffic — the monitor bus sustains at most one packet per two cycles. Runtime-disabling either class is safe (terminal entries auto-retire; see axi_monitor_reporter); alternatively, `cfg_axi_pkt_mask` drops the packets while keeping marking and counting. See `docs/user-guides/AXI_Monitor_Configuration_Guide.md`.

17.4.1.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit) select the event source (e.g. `3'b010` selects the `cfg_perf_enable` edge).
- `cfg_start_trigger` / `cfg_end_trigger` pulses fire the window directly from an engine or CSR.
- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

17.4.1.2 Utilization Counters (W channel)

Every in-window cycle is classified by the W-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	wvalid && wready	productive beat transferred
`perf_bp_cycles`	32	wvalid && !wready	back-pressure (data offered, consumer not ready)
`perf_starv_cycles`	32	!wvalid && wready	starvation (consumer ready, no valid data)
`perf_idle_cycles`	32	!wvalid && !wready	idle

The four buckets sum to `window_cycles`, so utilization = `perf_prod_cycles / window_cycles`.

17.4.1.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats × (1 << AxSIZE); AXIL is fixed at `AxSIZE=3'b010` (4 bytes)
`perf_burst_count`	32	AW (write-address) handshakes

AXI4-Lite note: every transaction is a single data beat (AWLEN is implicitly 0), so `perf_burst_count` counts AW handshakes = transactions and `perf_beat_count` equals the transaction count. Average burst length (`perf_beat_count / perf_burst_count`) is therefore always 1.

The perfmon config/status ports and the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control inputs have the same directions/widths and semantics as the **read-monitor port table** (only `perf_burst_count` counts AW handshakes here). As there, `cfg_debug_level`/`cfg_debug_mask`/`cfg_active_trans_threshold` are tied to constants internally and not exposed.

17.4.2 Monitor Backpressure (block_ready)

`block_ready` is a flow-control net inside the wrapper, and it IS brought out: `debug_block_ready` is an output port of this module (the `_cg` wrapper ties it off, so use the base module when you need the tap). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `s_axil_awready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `s_axil_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property,

ap_disabled_never_stalls; this module has no ifdef FORMAL block of its own, so here the guarantee rests on the gate expression above, not a proof).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS) (reserve = 4 for tables of 16 or more, 0 below; the function lives in monitor_common_pkg), and block_ready re-asserts at occupancy < MAX_TRANSACTIONS - (reserve - 1) – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size MAX_TRANSACTIONS to cover NUM_CHANNELS x per-channel outstanding plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's --unroll-count raised (default 64) in sim builds.

17.4.3 Address-Range Checker

Identical to **axil4_master_rd_mon** except the checker watches AW (write address) handshakes. The cfg_addr_* configuration inputs and monbus event encoding are the same. See the read monitor's Address-Range Checker section for full details.

17.4.4 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the cfg_* ports tied low filters nothing and looks like the feature is broken.

Address-range filter — ADDR_FILTER_ENABLE (bit, default 0) builds it.

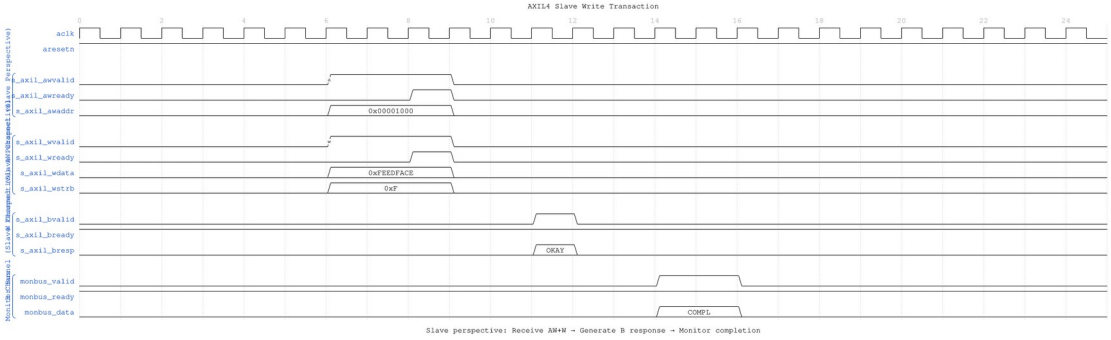
Port	Direction	Width	Description
cfg_addr_filter_enable	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
cfg_addr_filter_low	Input	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	Input	ADDR_WIDTH	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is what makes it safe to reprogram live. Contrast the address-range CHECKER above, which re-evaluates per command.

There is no runtime ID filter here. AXI-Lite has no transaction IDs, so this wrapper ties `cfg_id_filter_enable` / `cfg_id_match_base` / `cfg_id_match_count` off on the inner monitor rather than exposing them — a filter keyed on a field the protocol lacks has nothing to match against.

17.5 Waveforms

17.5.1 Scenario 1: Slave Write Transaction

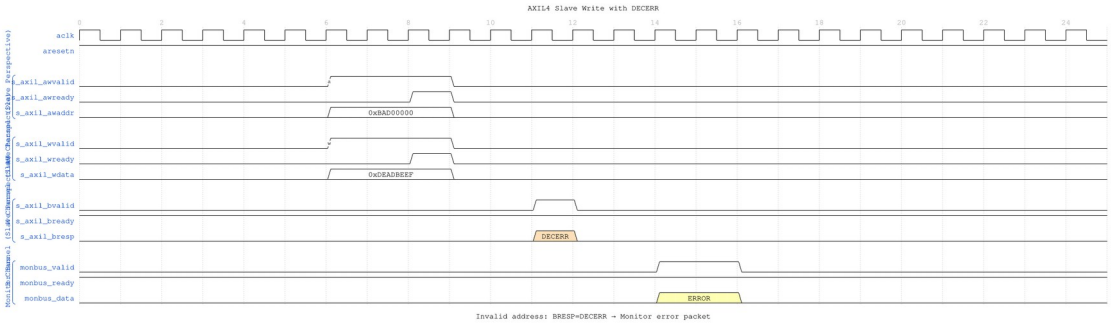


Slave Write Basic

WaveJSON: slave_write_basic_001.json

Key Observations: - Slave perspective: Receive AW+W simultaneously - Slave commits write to backend storage - Slave generates B response with BRESP=OKAY - Monitor tracks backend write latency

17.5.2 Scenario 2: Slave Write Error (DECERR)

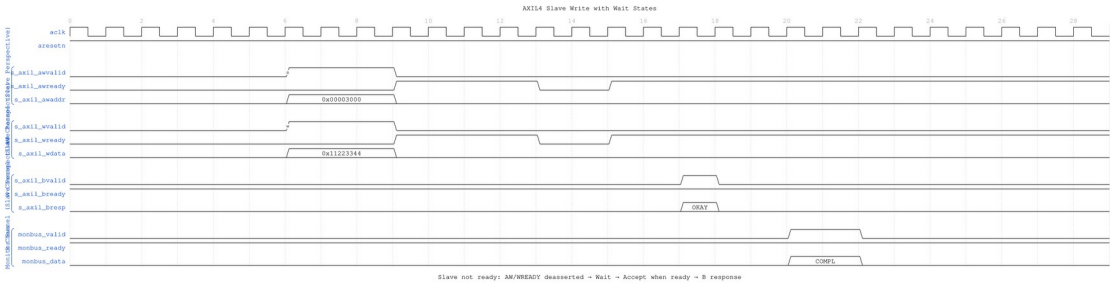


Slave Write DECERR

WaveJSON: slave_write_decerr_001.json

Key Observations: - Invalid address detected by slave - Write data received but not committed - Slave returns BRESP=DECERR - Monitor generates ERROR packet

17.5.3 Scenario 3: Slave Write with Wait States



Slave Write Wait

WaveJSON: slave_write_wait_001.json

Key Observations: - Slave not ready: AWREADY/WREADY deasserted - Master holds AW+W until slave accepts - Backend write processing delay - Monitor tracks full write latency including wait states

---

17.6 Usage Examples

```
axil4_slave_wr_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(2),
    .AGENT_ID(21),
    .MAX_TRANSACTIONS(8)
) u_axil_slave_wr_mon (
    // Slave AXIL write interfaces
    // Monitor configuration and bus
);
```

17.7 Timing Characteristics

17.7.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the

Parameter	Default	Channel
		AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

17.8 Design Notes

Monitor cost is not incremental. The transaction table is `bus_transaction_t` x `MAX_TRANSACTIONS`, and the reporter keeps a second full copy (`r_trans_table_local`), so a monitored interface is a multiple of the unmonitored one rather than a few percent on top. `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`perf_byte_count` scales with the bus width. `cmd_size` is derived as `$clog2(AXIL_DATA_WIDTH/8)`. It was hardwired to `3'b010` (4 bytes) until 2026-09-02, which halved every byte count on a 64-bit Lite bus without failing anything. `val/amba/test_axil_perf_byte_count.py` pins it at both legal widths.

Do not enable completion and performance packets together under load. The monitor bus sustains at most one packet per two cycles. Use `cfg_axi_pkt_mask` to drop a class while keeping its marking and counting.

The ID filter is inert and must stay that way. AXI4-Lite has no IDs, so this wrapper ties the monitor's ID inputs to zero; enabling `ID_FILTER_ENABLE` with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring rather than narrowing it.

17.9 Related Modules

- **axil4_slave_wr** - Base functional module
- **axil4_slave_rd_mon** - Read monitor counterpart

- **AXI4 Slave Write Mon** - Full AXI4 reference

Last Updated: 2026-07-19

17.10 Testing

`val/amba/test_axil4_slave_wr_mon.py` drives this module with the AXI4-Lite BFMs from `TBClasses/axil4`. It collects 3 parameter cases at the default `REG_LEVEL`. Run it with:

```
source env_python
pytest val/amba/test_axil4_slave_wr_mon.py -v
```

17.11 Navigation

- **← Back to AXIL4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

18 AXIL4 Slave Write Monitor (Clock-Gated)

Module: `axil4_slave_wr_mon_cg.sv` **Base Module:** axil4_slave_wr_mon **Location:** `rtl/amba/axil4/` **Status:** Partial — see Implementation Status

---

18.1 Overview

`axil4_slave_wr_mon_cg` wraps axil4_slave_wr_mon and adds a power-management control and status interface. All monitoring, filtering, address-range checking, and performance-monitoring behavior is that of the base module; see axil4_slave_wr_mon.md for the complete functional specification.

18.1.1 Implementation Status

This wrapper gates the monitor's clock for real. It instantiates `amba_clock_gate_ctrl`, and the base `axil4_slave_wr_mon` inside it is driven from `gated_aclk`, not from `aclk`.

What it does:

1. **Gates the clock.** `amba_clock_gate_ctrl` counts `cfg_cg_idle_count` idle cycles and stops `gated_aclk`, reporting on `cg_gating` (the gated clock is stopped) and `cg_idle` (no activity observed).
2. **Holds off the interfaces while gated.** `s_axil_awready`, `s_axil_wready` and `fub_axil_bready` are forced low under `cg_gating`, so nothing is accepted into a stopped clock.
3. **Masks the monbus valid while gated** (`monbus_valid = w_monbus_valid && !cg_gating`), which is what makes delivery exactly-once across a gating edge rather than a held valid replayed on wake.

Two earlier versions of this page were wrong in opposite directions, so both are worth naming. One described a `cg_cycles_saved` output, a `cfg_cg_idle_threshold` input and `ENABLE_CLOCK_GATING` / `CG_IDLE_CYCLES` parameters; none of those exist anywhere in `rtl/amba`. The other – the correction to the first – concluded that the wrapper therefore gates nothing and "will not reduce dynamic power". That was the more damaging error: it is false, and it steers an integrator away from the right part. The real controls are `cfg_cg_enable` and `cfg_cg_idle_count` (width `CG_IDLE_COUNT_WIDTH`); there is no cycles-saved counter of any kind.

Gating behaviour is asserted directly by `val/amba/test_mon_cg_gating.py` (phase 5 for the ready hold-off, phase 6 for exactly-once monbus delivery).

18.2 Parameters

In addition to all axil4_slave_wr_mon parameters (including `USE_MONITOR` and `N_ADDR_RANGES`):

Parameter	Type	Default	Description
`ACLK_MHZ`	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was unreachable through this wrapper until 2026-

Parameter	Type	Default	Description
			09-01.
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Lowest frequency the tick LUT must cover (dynamic-frequency builds).
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Highest frequency the tick LUT must cover.
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue. Required (=1) whenever NUM_BANKS > 1.
NUM_BANKS	int	1	Transaction-table banking. >**1 on this WRITE monitor requires USE_WDATA_ORDER_Q=1** – axi_monitor_trans_mgr fails elaboration otherwise (the WID-less fallback double-counts one W beat across banks).
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves cfg_addr_filter_enable low filters nothing and looks broken.
CG_IDLE_COUNT_WIDTH	int	4	Width of cfg_cg_idle_count; sets the longest programmable idle threshold

Parameter	Type	Default	Description
ADD_PIPELINE_STAGE	bit	0	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing.

There are no CG_GATE_MONITOR, CG_GATE_REPORTER, or CG_GATE_TIMERS parameters, and no independent gating domains. Earlier revisions of this document described such a scheme; it was never implemented.

18.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXIL_ADDR_WIDTH
DW	AXIL_DATA_WIDTH

18.3 Ports

18.3.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
cfg_addr_filter_enable	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever ADDR_FILTER_ENABLE says
cfg_addr_	Input	ADDR_WIDT	Window base, inclusive

Port	Direction	Width	Description
filter_low		H	
cfg_addr_filter_high	Input	ADDR_WIDTH	Window limit, inclusive

There is no runtime ID filter on AXI4-Lite: the protocol has no IDs to match.

Base-module ports are forwarded unchanged EXCEPT debug_block_ready, which this wrapper does not bring out (use the base module when you need that tap). That includes the cam_clear control input (Input, 1) - synchronous clear of the monitor transaction CAM, driven from the harness clear control bit, e.g. CTRL[4]. The wrapper adds four ports:

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Arms clock gating. It reaches amba_clock_gate_ctrl only – cfg_monitor_enable is forwarded to the inner monitor untouched, so 0 here does NOT disable the monitor, it just leaves the clock free-running.
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before the clock gates. The clock stops cfg_cg_idle_count + 2 cycles after the last bus activity (one extra for the r_wakeup flop).
cg_gating	Output	1	High while the clock is gated.
cg_idle	Output	1	High while the activity terms are quiet.

The base module's busy output remains available on this wrapper.

18.4 Functional Description

18.4.1 Performance Monitoring

The wrapper forwards the base module's full performance-monitoring interface to `axi_monitor_base` **unchanged** — the power-management interface neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for axil4_slave_wr_mon:

- **Config inputs:** `cfg_perf_enable`, `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Status / counters:** `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`

The completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) and the synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) are likewise forwarded unchanged. The utilization buckets watch the **W** (write-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AW handshakes = transactions.

> Note that `cfg_cg_enable = 0` does NOT disable the monitor. It only disarms clock gating, leaving the inner monitor on a free-running clock and behaving exactly like the base module. Use `cfg_monitor_enable = 0` to actually stop monitoring.

18.5 Usage Examples

```
axil4_slave_wr_mon_cg #(
    // Base module parameters (see axil4_slave_wr_mon.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2),

    // Monitor parameters
    .UNIT_ID(8'h01),
    .AGENT_ID(16'h000A),
    .MAX_TRANSACTIONS(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
```

```
    // Power-management interface
    .cfg_cg_enable(1'b1),           // arm gating; 0 = clock free-
runs
    .cfg_cg_idle_count(4'd4),       // idle cycles before the clock
stops
    .cg_gating(cg_gating),          // high while the gated clock is
stopped
    .cg_idle(cg_idle),

    // ... all other ports same as axil4_slave_wr_mon
);
```

18.6 Timing Characteristics

18.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

18.7 Design Notes

Monitor cost is not incremental. The transaction table is `bus_transaction_t` x `MAX_TRANSACTIONS`, and the reporter keeps a second full copy (`r_trans_table_local`), so a monitored interface is a multiple of the unmonitored one rather than a few percent on top. `MAX_TRANSACTIONS` is the knob and the cost is linear in it.

`perf_byte_count` scales with the bus width. `cmd_size` is derived as `$clog2(AXIL_DATA_WIDTH/8)`. It was hardwired to `3'b010` (4 bytes) until 2026-09-02, which halved every byte count on a 64-bit Lite bus without failing anything. `val/amba/test_axil_perf_byte_count.py` pins it at both legal widths.

Do not enable completion and performance packets together under load. The monitor bus sustains at most one packet per two cycles. Use `cfg_axi_pkt_mask` to drop a class while keeping its marking and counting.

The ID filter is inert and must stay that way. AXI4-Lite has no IDs, so this wrapper ties the monitor's ID inputs to zero; enabling `ID_FILTER_ENABLE` with an `ID_MATCH_BASE` above 0 makes `id_owned(0)` false for every transaction and drops ALL monitoring rather than narrowing it.

18.8 Related Modules

- **axil4_slave_wr_mon** - Base module (functional specification)
- **axil4_slave_rd_mon_cg** - Companion monitor wrapper
- **axi_monitor_base** - Core monitoring infrastructure
- **axi_monitor_filtered** - Filtering capabilities
- **AXIL4 Clock-Gated Variants Guide** - The transport-level `_cg` modules, which do perform real clock gating

---

18.9 Testing

A clock IS gated here – that is the wrapper's entire purpose. `amba_clock_gate_ctrl` drives the inner monitor's clock, and it stops when the activity terms go quiet for `cfg_cg_idle_count` cycles.

`cfg_cg_enable` arms that gating. It is NOT a monitor kill-switch: with it low the clock simply free-runs and the monitor behaves exactly like the base module, packets included. Drive it low for any test that wants the base module's timing without gating effects; drive it high to exercise the gating itself, and expect the counters and any trigger pulse to be lost while the clock is stopped (see the warning under Performance Monitoring).

Last Updated: 2026-07-19

18.10 Navigation

- ← Back to Base Module
- ← Back to AXIL4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index